

Universidad ORT Uruguay
Facultad de Ingeniería

Documentación

OBLIGATORIO 1

Base de datos no relacionales

Julieta López - 295568
Florencia Mendoza - 285422
Natalia Franco - 310914

Tutores :
Juan Gabito
Antonella Musso

Repositorio:
<https://github.com/NataliaFrancoJ/BDNR.git>

2025

Índice

Introducción.....	4
Requerimiento 1.....	5
Tienda virtual y suscripciones.....	5
Motor de base de datos.....	5
Requerimientos no funcionales.....	5
Modelo de datos previsto.....	6
Principales patrones de acceso identificados.....	7
Principales ventajas, desventajas y trade-offs.....	8
Analítica y seguimiento de actividad.....	10
Motor de base de datos.....	10
Modelo de datos previsto.....	11
Principales patrones de acceso.....	11
Principales ventajas, desventajas y trade-offs.....	12
Privacidad y seguridad.....	13
Motor de base de datos.....	13
Requerimientos no funcionales.....	13
Modelo de datos previsto.....	14
Principales patrones de acceso identificados.....	15
Principales ventajas, desventajas y trade-offs.....	16
Gestión de usuarios y perfiles.....	18
Motor de base de datos.....	18
Requerimientos no funcionales.....	18
Modelo de datos previsto.....	19
Principales patrones de acceso identificados.....	22
Principales ventajas, desventajas y trade-offs.....	23
Sistemas de lecciones y ejercicios.....	25
Motor de base de datos.....	25
Requerimientos no funcionales.....	25
Modelo de datos previsto.....	26
Principales patrones de acceso identificados.....	28
Principales ventajas, desventajas y trade-offs.....	28
Motor de recomendaciones.....	30
Motor de base de datos.....	30
Modelo de datos previsto.....	30
Principales patrones de acceso identificados.....	32
Principales ventajas, desventajas y trade-offs.....	34
Foros y comunidad.....	35
Motor de base de datos.....	35
Requerimientos no funcionales.....	35
Modelo de datos previsto.....	36
Principales patrones de acceso identificados.....	37
Principales ventajas, desventajas y trade-offs.....	38

Sistemas de notificaciones.....	40
Motor de base de datos.....	40
Requerimientos no funcionales.....	40
Modelo de datos previsto.....	41
Principales patrones de acceso identificados.....	41
Principales ventajas, desventajas y trade-offs.....	42
Requerimiento 2.....	44
Detalle técnico:.....	45
Tipos de datos.....	45
Representación física:.....	47
Implementación.....	49
Requerimiento 3.....	50
Requerimiento 4.....	54
Implementación de Gestión de usuarios y perfiles.....	54
Flexibilidad del modelo.....	56
Pasos para ejecutar la implementación.....	56
Implementación de Motor de Recomendaciones.....	59
Cambios de la primera entrega.....	62
Bibliografía.....	63
Apartado de Inteligencia Artificial.....	64

Introducción

En este trabajo del curso Bases de Datos No Relacionales analizamos el caso de Duolingo, una aplicación muy conocida para aprender idiomas que maneja millones de usuarios y una enorme cantidad de datos todos los días.

El objetivo del obligatorio es entender cómo se pueden aplicar distintos tipos de bases de datos en un sistema real como este, eligiendo la tecnología que mejor se ajuste a las necesidades de cada subsistema.

Explicamos qué motores de base de datos se podrían usar en los diferentes subsistemas de Duolingo, por qué los elegimos y qué ventajas o desventajas tiene cada uno.

También trabajamos con un formato de almacenamiento columnar, en nuestro caso Apache Arrow, para analizar cómo se organizan, cargan y procesan los datos de manera eficiente. Se detalla su funcionamiento técnico, la forma en que representa la información en memoria y las ventajas que ofrece en términos de rendimiento y compresión.

Además, modelamos las entidades usadas para el subsistema “Motor de recomendaciones” según la base de datos seleccionada y las necesidades planteadas.

Por último, implementamos el almacenamiento para los subsistemas “Gestión de usuarios y perfiles” y “Motor de recomendaciones”

El link al repositorio donde se encuentran las implementaciones es:

<https://github.com/NataliaFrancoJ/BDNR.git>

Requerimiento 1

Tienda virtual y suscripciones

Motor de base de datos

Para el subsistema Tienda virtual y suscripciones se usará PostgreSQL, una base de datos relacional clásica que garantiza la integridad y consistencia de las transacciones financieras. Su soporte para operaciones ACID y su control avanzado de concurrencia permiten manejar de forma segura las suscripciones, renovaciones y pagos de los usuarios, asegurando confiabilidad y estabilidad en la gestión de la información.

Requerimientos no funcionales

A continuación, enunciamos los requerimientos no funcionales que identificamos y en los que basamos la selección de usar PostgreSQL:

Confiabilidad y consistencia de los datos: *“El sistema permite a los usuarios suscribirse, renovar o cancelar su membresía en cualquier momento, manteniendo un registro detallado de todas las transacciones realizadas.”*

Como se trata de pagos y suscripciones, los datos tienen que ser exactos. No puede haber registros duplicados ni operaciones a medio completar. PostgreSQL garantiza esto gracias a su soporte de transacciones ACID, lo que asegura que cada operación se guarde correctamente o no se aplique si ocurre un error. Así se evita perder o corromper información financiera.

Integridad referencial: *“El sistema permite a los usuarios suscribirse, renovar o cancelar su membresía en cualquier momento, manteniendo un registro detallado de todas las transacciones realizadas. Además, el historial de compras proporciona información sobre fechas de pago, duración de la suscripción y métodos de pago utilizados, permitiendo un fácil acceso a los datos en caso de consultas o reembolsos.”*

El requerimiento deja claro que todo está conectado: usuarios, pagos, fechas, métodos, etc. PostgreSQL permite manejar bien esas relaciones con claves foráneas y restricciones. De esa forma, si un usuario se elimina o cambia, sus transacciones siguen estando bien vinculadas. No se rompe la estructura ni se mezclan datos.

Control de concurrencia: *“El sistema permite a los usuarios suscribirse, renovar o cancelar su membresía en cualquier momento...”*

Esto significa que muchos usuarios pueden estar haciendo operaciones al mismo tiempo. PostgreSQL usa un sistema de control de concurrencia (MVCC) que deja que varias personas modifiquen o lean datos sin interferir entre sí. Así, todos pueden comprar o renovar al mismo tiempo sin que el sistema se bloquee.

Seguridad y protección de datos: *“Para garantizar una experiencia segura y confiable, el sistema de pagos emplea protocolos de seguridad avanzados, protegiendo la información financiera de los usuarios.”*

Como se manejan datos de pago, la seguridad es clave. PostgreSQL tiene autenticación por roles, cifrado de conexión y control de acceso a nivel de usuario. Esto ayuda a cumplir con el requerimiento, protegiendo los datos financieros y evitando accesos no autorizados.

Disponibilidad y estabilidad: *“Las renovaciones automáticas y las notificaciones sobre vencimientos ayudan a evitar interrupciones en el acceso a las funciones premium...”*

El sistema tiene que estar disponible todo el tiempo, sin interrupciones. PostgreSQL permite configurar réplicas y copias de seguridad automáticas, lo que asegura que, aunque algo falle, la base siga funcionando y no se pierdan datos.

Auditoría y trazabilidad de operaciones: *“...manteniendo un registro detallado de todas las transacciones realizadas. Además, el historial de compras proporciona información sobre fechas de pago...”*

PostgreSQL puede registrar automáticamente cada cambio mediante triggers o funciones, guardando un historial de operaciones. Esto permite ver, por ejemplo, qué usuario pagó, qué suscripción modificó y en qué fecha, algo esencial para consultas o reclamos.

Escalabilidad y rendimiento estable: *“La correcta gestión de las cuentas premium y sus transacciones es esencial para mantener la satisfacción del usuario y la estabilidad del modelo de negocio de Duolingo...”*

PostgreSQL puede crecer fácilmente si la cantidad de usuarios o transacciones aumenta. Permite optimizar consultas con índices y vistas, y escalar con réplicas de lectura. Esto garantiza que, aunque la cantidad de operaciones crezca, el sistema siga funcionando rápido y sin afectar la experiencia del usuario.

Modelo de datos previsto

En este subsistema, el modelo de datos se basa en información estructurada, persistente y fuertemente relacionada, ya que se necesita mantener un registro confiable de las compras, suscripciones y transacciones de los usuarios.

El modelo sigue una estructura relacional clásica, donde cada entidad representa un aspecto fundamental de la tienda: los productos disponibles, las órdenes de compra, los pagos realizados y las suscripciones activas o históricas de los usuarios.

Las claves primarias garantizan la unicidad de los registros, mientras que las claves foráneas aseguran la coherencia entre las distintas entidades.

A continuación se detallan las principales tablas y sus campos asociados:

Usuario (usuario)

- id_usuario: Identificador único del usuario
- email: Correo electrónico del usuario
- nombre_usuario: Nombre o alias dentro de la plataforma
- fecha_creacion: Fecha en que se registró el usuario

Método de pago (metodo_pago)

- id_metodo_pago: Identificador del método de pago
- id_usuario: Usuario propietario del método (FK: usuario.id_usuario)
- tipo: Tipo de método de pago (tarjeta, PayPal, etc.)
- token_pago: Token cifrado que representa el método ante el procesador
- activo: Si el método está disponible para usar

Suscripción (suscripcion)

- id_suscripcion: Identificador único de la suscripción
- id_usuario: Usuario que posee la suscripción (FK: usuario.id_usuario)
- tipo_plan: Tipo de plan adquirido (Super Duolingo o Súper Familiar)
- fecha_inicio: Fecha en que comenzó la suscripción
- fecha_fin: Fecha de vencimiento o finalización de la suscripción
- estado: Estado actual (activa, vencida, cancelada)
- renovacion_automatica: Si la suscripción se renueva sola al vencer
- id_metodo_pago: Método de pago asociado (FK: metodo_pago.id_metodo_pago)
- monto_mensual: Precio mensual del plan
- moneda: Moneda usada (dólares, pesos, etc)
- fecha_creacion: Cuándo se generó el registro
- fecha_actualizacion: Última vez que se modificó el estado

Transacción (transaccion)

Registra todas las operaciones financieras relacionadas con las suscripciones: pagos, renovaciones, cancelaciones, reembolsos.

- id_transaccion: Identificador único de la transaccion
- id_usuario: Usuario que realizó la transacción (FK: usuario.id_usuario)
- id_suscripcion: Suscripción afectada (FK: suscripcion.id_suscripcion)
- fecha_transaccion: Fecha y hora exacta del evento
- tipo_transaccion: Tipo de operación (pago, renovación, cancelación, reembolso)
- estado: Estado del proceso (pendiente, completada, fallida)
- detalle: Observaciones o descripción de la operación

Principales patrones de acceso identificados

Para este subsistema, los principales patrones de acceso tienen que ver con la persistencia de los datos, las transacciones seguras y las consultas relacionales optimizadas.

Consultas transaccionales y consistentes

Las operaciones principales del sistema requieren lecturas y escrituras bajo garantía ACID,

ya que involucran dinero real y estados de suscripción.

Cada pago, renovación o cancelación debe registrarse de forma atómica, asegurando que los datos sean consistentes incluso ante fallos.

Por ejemplo, cuando un usuario renueva su membresía, se registra simultáneamente la nueva fecha de fin en la tabla suscripción y la transacción correspondiente en transacción.

Lecturas frecuentes y filtradas

El sistema realiza muchas consultas de lectura para mostrar al usuario el estado de su suscripción o su historial de pagos.

Estas lecturas suelen usar filtros por `id_usuario`, `estado` o `fecha_transaccion`, y requieren índices bien definidos para mantener un tiempo de respuesta bajo.

Por ejemplo, consultar si una suscripción está activa o recuperar todas las transacciones completadas de un usuario.

Escrituras controladas y seguras

Cada vez que un usuario realiza un pago o modifica su plan, se insertan o actualizan registros en las tablas suscripcion y transaccion. Estas operaciones deben ejecutarse dentro de transacciones para evitar inconsistencias entre los montos, estados y fechas de renovación.

Por ejemplo, al cancelar una suscripción, el sistema cambia el estado a “cancelada” y crea un registro de transacción tipo reembolso en la misma operación.

Actualizaciones periódicas programadas

El sistema realiza procesos automáticos que revisan suscripciones con renovación habilitada o próximas a vencer. Estos procesos ejecutan lecturas masivas y actualizaciones en lote, aplicando nuevas fechas de pago o cambiando estados a “vencida” si corresponde.

Integridad referencial y validación cruzada

El subsistema consulta otras tablas (como `usuario` o `metodo_pago`) para validar la información antes de confirmar una operación. Este patrón garantiza que no existan transacciones que no están asociadas a una suscripción o a un usuario, o métodos de pago inactivos. Por ejemplo, antes de registrar un cobro, el sistema verifica que el método de pago esté activo y asociado al usuario correcto.

Principales ventajas, desventajas y trade-offs

Ventajas:

- **Consistencia y confiabilidad:** PostgreSQL ofrece un modelo transaccional completo (ACID), ideal para manejar pagos, renovaciones y cambios de estado en las suscripciones sin riesgo de pérdida o duplicación de información. Cada operación se confirma de forma atómica, lo que asegura que los datos sean siempre coherentes.
- **Integridad referencial:** El uso de claves foráneas y restricciones garantiza que no existan transacciones sin referencia válida, evitando errores lógicos entre usuarios,

métodos de pago y suscripciones. Esto permite mantener una trazabilidad clara de cada operación.

- **Soporte para consultas complejas:** El motor relacional permite ejecutar fácilmente consultas filtradas, estadísticas o agregaciones, como obtener ingresos totales por mes, cantidad de suscriptores activos o historial de pagos de un usuario, sin necesidad de sistemas externos.
- **Seguridad y control de acceso:** PostgreSQL ofrece mecanismos de roles, cifrado y control de privilegios que protegen los datos sensibles. En particular, los tokens y métodos de pago pueden almacenarse cifrados, reduciendo el riesgo de exposición.
- **Escalabilidad vertical y replicación:** El sistema puede crecer en capacidad (CPU, memoria, almacenamiento) y también replicar datos hacia nodos de lectura para distribuir la carga entre procesos analíticos y operacionales sin afectar el rendimiento del servicio principal.

Desventajas:

- **Escalabilidad horizontal limitada:** Al ser un motor relacional, PostgreSQL no escala de forma tan simple como las bases distribuidas. Para volúmenes masivos de usuarios o transacciones, se requiere una arquitectura con réplicas, particiones o sharding manual.
- **Escrituras costosas en picos de carga:** Durante renovaciones automáticas simultáneas, el volumen de inserciones puede generar bloqueos o esperas en las transacciones. Se necesita un diseño cuidadoso de índices y procesos batch para evitar cuellos de botella.
- **Mantenimiento más complejo:** La administración de backups, logs de transacciones y ajustes de rendimiento requiere conocimiento técnico. En sistemas con mucha actividad financiera, la configuración de autovacuum, índices y checkpoints es crítica.

Trade-offs

La elección de PostgreSQL implica priorizar consistencia y seguridad por encima de la elasticidad. A diferencia de una base distribuida o en memoria, este enfoque garantiza la fiabilidad de los pagos y la trazabilidad de las suscripciones, pero a costa de una menor flexibilidad ante crecimientos súbitos de carga.

El modelo relacional resulta ideal para la estabilidad operativa del negocio, aunque podría complementarse en el futuro con soluciones de caché o colas de eventos para mejorar el rendimiento en escenarios de alta concurrencia.

Analítica y seguimiento de actividad

Motor de base de datos

Para el subsistema Analítica y seguimiento de actividad se usará Vertica, la cual está diseñada para manejar y procesar grandes volúmenes de datos de forma eficiente. Su arquitectura se basa en el almacenamiento columnar, lo que permite un acceso más rápido y un menor tiempo de lectura para las consultas.

Requerimientos no funcionales

Identificación de los requerimientos no funcionales en los que nos basamos para seleccionar Vertica:

Volumen y velocidad de ingesta: *“registra de manera continua diversos eventos durante la interacción del usuario”.*

Dado que entran muchos eventos todo el tiempo necesitamos absorber un flujo constante y grande.

Vertica nos permite cargar datos en bloques pequeños(micro batches), guardar por columnas (ocupa menos) y particionar por fecha.

Consultas rápidas: *“El análisis de estos datos permite ajustar la dificultad, optimizar la progresión de los cursos y ofrecer sugerencias personalizadas.”*

Necesitamos respuestas en segundos/minutos para ajustar dificultad, progresión y personalización.

Vertica al ser columnas, lee solo las columnas necesarias y calcula agregación muy rápido.

Escalabilidad horizontal: *“A través del análisis de tendencias y comportamientos de los usuarios, Duolingo puede identificar qué tipos de ejercicios son más efectivos, en qué momentos los estudiantes tienden a abandonar una lección y cómo mantener su motivación a lo largo del curso.*

Gracias a este enfoque basado en datos, la plataforma evoluciona constantemente “

Debemos crecer en datos/usuarios sin perder rendimiento. Vertica al ser distribuida agrega nodos y reparte datos/cálculos para mantener la velocidad.

Alta concurrencia de lectura: *“Evaluar la efectividad de nuevas metodologías de enseñanza y mejorar la experiencia general”.* (múltiples áreas mirando resultados al mismo tiempo)

Vertica soporta muchas lecturas porque las consultas de solo lecturas, usa columnas y puede tener vistas que sirven a cada tablero sin reprocesar todo.

Costo-eficiencia: *“Registra.. intentos, tiempo de uso y patrones de error más comunes”*

Se necesita guardar mucho y consultar rápido pero gastando menos. Vertica comprime fuerte y lee pocas columnas; con particiones por tiempo evitas leer datos viejos cuando no hacen falta.

Alta disponibilidad y resiliencia: *“Gracias a este enfoque basado en datos, la plataforma evoluciona constantemente...”*

Evitar caídas que frenen decisiones, que el servicio pueda continuar. Vertica al ser distribuida y replicada, tolera fallas y se recupera sin rehacer todo.

Compatibilidad con SQL y ecosistema BI: *“Identificar que tipos de ejercicios son más efectivos, en qué momento los estudiantes abandonan..... y cómo mantener su motivación”*

Vertica entiende SQL estándar, tiene funciones de ventana, se conecta fácil a herramientas de BI.

Modelo de datos previsto

A continuación, mostramos cómo se van a organizar los datos para esta sección.

Para el modelo de datos, adoptaremos un esquema estrella cuyo núcleo es la tabla de hechos *fact_evento*, donde cada fila representa un evento atómico generado por un usuario (ejemplo, intento de ejercicio) e incluye las claves *event_id*, *event_time* (timestamp) y *event_date* (date), además de *user_id*, *course_id*, *lesson_id* y *event_type*, junto con métricas como duración, resultado y puntaje. El modelo se completa con las dimensiones de usuario, curso, lección y tiempo, que aportan el contexto para segmentar el análisis.

La tabla se particiona por días o por mes para que consultar por rangos de tiempo sea rápido, y se ordena/segmenta por tiempo del evento y un id de usuario para optimizar cálculos por usuario y series de tiempo. En el diagrama también vamos a visualizar las dimensiones, la dimensión del usuario, del curso, de la lección y el tiempo.

Este diseño funciona bien en Vertica porque es columnar y permite leer solo las columnas necesarias, comprimir mucho y unir con dimensiones sin perder rendimiento.

Principales patrones de acceso

Consultas por rango temporal

Las consultas más comunes serán por tiempo(últimas 24 horas, 7 días, etc) para ver tasas de error, tiempo promedio por intento y evolución semanal.

Vertica nos permite particionar por fecha para poder evitar leer datos viejos.

Agregaciones y funciones de ventana

para percentiles, promedios móviles y rachas por usuario

Análisis Cohortes y funnels

Para entender retención y puntos de abandono. Con Vertica tendremos un buen soporte de joins y ventanas sobre millones de filas.

Personalización y sugerencias

Para poder detectar usuarios con errores en una estructura gramatical y enviar ejercicios reforzados.

Vertica filtra por columnas específicas y calcula métricas por usuario/lección sin leer todo.

A/B testing

comparar puntaje/tiempo/abandono entre variante A y B.

Vertica: agrupa y compara grandes volúmenes por variante de forma paralela.

Dashboards BI con alta concurrencia

Panel de efectividad por curso, mapa de calor de abandono por hora.

Por qué Vertica: consultas de solo lectura sobre proyecciones/segmentaciones optimizadas.

Principales ventajas, desventajas y trade-offs

Ventajas:

- **Agregación rápida:** Vertica guarda los datos “por columnas” y trabaja en paralelo, así que calcular promedios, conteos o percentiles sobre millones de filas es mucho más rápido que en una base tradicional.
- **Eficiencia de almacenamiento:** al comprimir columnas, los datos pesan menos y el almacenamiento no es tan costoso.
- **Particionado por tiempo:** nos permite realizar cálculos recientes y rápidos, facilita la retención/archivado. Es más fácil archivar lo viejo.
- **Segmentación:** podemos guardar los datos “ordenados para leer mejor”. Con la segmentación podemos guardar los eventos ya pensados para las consultas típicas y así consultar más rápido.
- **Usa SQL estándar y se conecta a BI:** se integra con herramientas y skills existentes.

Desventajas:

- **Operación:** hay que operar como un clúster, tiene varios nodos; eso implica monitoreo, backups y balanceo.
- **Escrituras intensivas:** no es un motor OLTP; las escrituras punto a punto muy pequeñas no son su fuerte.
- **Coste inicial:** mayor “setup” que una base relacional tradicional. Al principio hay que definir particiones y proyecciones.
- **Cambios de esquema:** en tablas enormes hay que planificarlos.

Trade-offs

Optimizamos lecturas analíticas más rápidas y menos costosas a gran escala a cambio de no usarlas para transacciones finas en tiempo real. Para eventos de aplicación en tiempo real/OLTP usamos otra base operativa; Vertica es el almacén analítico.

Privacidad y seguridad

Motor de base de datos

Para el subsistema de Privacidad y seguridad usaremos Redis, la cual es una base de datos clave-valor en memoria, altamente eficiente y orientada al rendimiento. Es ideal para escenarios donde se requieren lecturas y escrituras de baja latencia, manejo de tokens efímeros, cachés cifradas, y control de sesiones seguras.

Requerimientos no funcionales

A continuación, enunciamos los requerimientos no funcionales que identificamos y en los que basamos la selección de usar Redis:

Registro de operaciones para auditoría: *“Cada operación queda registrada para auditoría...”*

Redis se puede usar con cifrado y autenticación, así que se adapta al requerimiento de proteger los datos tanto mientras están guardados como mientras se envían.

Consistencia controlada: *“solo los perfiles autorizados pueden interactuar con los datos, minimizando la exposición de información.”*

Redis prioriza la velocidad sobre la consistencia absoluta, pero como los accesos están controlados y auditados, ese tipo de consistencia es suficiente para este caso.

Alta disponibilidad: *“Estas operaciones se aplican en línea, de forma inmediata, sin afectar la disponibilidad general de la plataforma ni la experiencia de aprendizaje.”*

Redis se puede configurar con replicación y respaldo, así que si un nodo falla, otro sigue funcionando. Eso cumple con lo que se pide, que nada de esto afecte el funcionamiento de Duolingo.

Manejo de claves efímeras: *“...la información crítica se maneja con claves efímeras y mecanismos de expiración automática.”*

Redis tiene una función llamada TTL (Time To Live) que borra automáticamente los datos después de cierto tiempo. Esto es ideal para guardar tokens temporales o claves que se destruyen solas cuando ya no sirven.

Seguridad y cifrado: *“Para equilibrar seguridad y velocidad, los datos se cifran en reposo y en tránsito...”*

Redis tiene una función llamada TTL (Time To Live) que borra automáticamente los datos después de cierto tiempo. Esto es ideal para guardar tokens temporales o claves que se destruyen solas cuando ya no sirven.

Actualización inmediata: *“Estas operaciones se aplican en línea, de forma inmediata, sin afectar la disponibilidad general de la plataforma ni la experiencia de aprendizaje.”*

Redis escribe en memoria, así que los cambios se aplican enseguida, como se pide en el requerimiento: las operaciones deben ser inmediatas y no interrumpir al usuario.

Baja latencia: *“Esto permite mantener lecturas de baja latencia para la mayoría de los accesos mediante cachés de solo lectura...”*

Redis guarda los datos en memoria, por lo cual, al hacer una consulta, el tiempo de respuesta es inmediato. En el requerimiento se pide que las lecturas deben ser de baja latencia, o sea que sean rápidas y sin demora al consultar preferencias y validaciones.

Uso de cachés de solo lectura: *“Esto permite mantener lecturas de baja latencia para la mayoría de los accesos mediante cachés de solo lectura...”*

Redis se usa mucho como caché. Puede guardar temporalmente datos no sensibles para no consultar otras bases más lentas.

Modelo de datos previsto

En este subsistema, el modelo de datos se basa principalmente en información temporal, segura y accesible de inmediato. Como Redis trabaja con claves y valores, no se manejan tablas ni relaciones tradicionales, sino distintos conjuntos de datos pensados para cumplir funciones específicas dentro del sistema.

Por un lado, están las **sesiones de usuario**, donde se guarda información de autenticación como el token de acceso, el rol del usuario y los permisos que tiene habilitados. Cada sesión se crea con un tiempo de vida definido (TTL), de modo que expira automáticamente después de un período determinado, por ejemplo, una hora. Esto permite que las sesiones se gestionen de forma segura y sin necesidad de limpiarlas manualmente.

También se almacenan las **preferencias de privacidad** de cada usuario. Acá se guardaría por ejemplo, si el usuario quiere mostrar su foto de perfil, compartir su progreso o recibir solicitudes de amistad. Estos datos se mantienen de forma persistente (sin expiración automática), y se guardan generalmente en formato JSON para poder leerlos y actualizarlos fácilmente desde la aplicación.

Otro conjunto importante son los **registros de auditoría**. Estos sirven para dejar constancia de las acciones relacionadas con la información sensible, como quién modificó una preferencia o cuándo se consultó un dato privado. Estos registros se guardan ordenados por tiempo y solo se conservan durante unos días (por ejemplo, una semana), después de lo cual se eliminan o se trasladan a otro sistema de almacenamiento más permanente.

Por último, el modelo incluye las **claves efímeras**, que son valores temporales usados para operaciones críticas como el cifrado o la validación de acciones sensibles (por ejemplo, confirmar la eliminación de una cuenta). Estas claves tienen una vida útil muy corta, de apenas unos minutos, y Redis se encarga de destruirlas automáticamente una vez que expiran.

A continuación, detallamos las claves (key) y sus campos asociados:

Sesión de usuario (session:{userId})

- token: Identificador único generado al iniciar sesión (autenticación)
- userId: Identificador del usuario

- rol: Rol del usuario dentro de la plataforma (admin, estudiante, etc.)
- permissions: Lista de permisos habilitados para esa sesión
- ipAddress: Dirección IP desde donde se inició la sesión
- createdAt: Fecha/hora de creación de la sesión
- expiresAt: Tiempo de vida o expiración automática

Preferencias de privacidad (privacy:preferences:{userId})

- userId: Identificador del usuario
- shareProgress: Si el usuario comparte o no su progreso públicamente
- showProfilePicture: Visibilidad de la foto de perfil
- allowFriendRequests: Si acepta solicitudes de amistad
- dataRetention: Tiempo que desea mantener sus datos
- lastUpdated: Última fecha en que modificó las preferencias

Registros de auditoría (audit:log:{operationId})

- operationId: Identificador único de la acción registrada
- userId: Usuario que realizó la acción
- action: Tipo de acción (lectura, actualización, eliminación)
- resource: Qué dato fue afectado
- timestamp: Fecha y hora exacta del evento
- ipAddress: Desde qué IP se realizó la acción
- status: Resultado de la operación

Clave efímera (key:ephemeral:{tokenId})

- tokenId: Identificador de la clave temporal
- keyValue: Valor cifrado o secreto temporal
- purpose: Para qué se usa la clave (ej: validar eliminación de cuenta)
- createdAt: Cuándo se generó la clave
- TTL: Tiempo de vida antes de destruirse

Principales patrones de acceso identificados

Para este subsistema los principales patrones de acceso tienen que ver con cómo el sistema maneja la información sensible y temporal de los usuarios.

Lecturas rápidas

El sistema realiza constantemente lecturas de baja latencia para validar permisos, sesiones y configuraciones de privacidad.

Estos accesos deben ser inmediatos porque se ejecutan mientras el usuario navega o realiza acciones.

Por ejemplo, verificar si un token de sesión es válido o consultar si el usuario permite mostrar su perfil o compartir su progreso.

En Redis esto se logra gracias a que todos los datos están en memoria y las lecturas son instantáneas.

Modificaciones inmediatas

Los usuarios pueden cambiar sus preferencias de privacidad o cerrar sesión en cualquier momento.

Por eso el patrón incluye escrituras rápidas que deben reflejarse al instante en el sistema, sin afectar la disponibilidad general.

Por ejemplo, actualizar la preferencia de compartir progreso o registrar el cierre de sesión.

Expiración automática de datos (TTL)

Los datos sensibles, como tokens o claves efímeras, tienen un tiempo de vida limitado.

Este patrón es fundamental porque el sistema necesita borrar automáticamente información una vez que deja de ser válida.

Por ejemplo, asignar expiración a una sesión.

Registro temporal de operaciones

Cada acción sobre datos sensibles se registra para auditoría, pero solo se conserva por un tiempo limitado.

Este patrón combina inserciones frecuentes con eliminación programada.

Cacheo temporal

Para reducir carga en otros microservicios, Redis mantiene copias en memoria de datos que se consultan con frecuencia.

Esto evita consultar bases más lentas o sistemas externos

Principales ventajas, desventajas y trade-offs

Ventajas:

- **Velocidad y baja latencia:** Redis trabaja completamente en memoria, por lo que las operaciones de lectura y escritura se realizan casi al instante.
- **TTL:** Redis permite definir tiempos de vida por clave. Esto encaja perfectamente con los requerimientos del subsistema, donde las sesiones, tokens y claves temporales deben expirar automáticamente, sin procesos manuales ni sobrecarga del sistema.
- **Alta disponibilidad y escalabilidad:** Redis soporta replicación y particionamiento (Redis Cluster), lo que facilita manejar la gran cantidad de usuarios activos de Duolingo y mantener disponibilidad incluso ante fallos.
- **Integración:** Redis es muy liviano y se integra fácilmente con microservicios mediante APIs o librerías nativas. Esto es ideal para una arquitectura distribuida como la de Duolingo, donde cada subsistema tiene su propia base de datos.
- **Soporte de seguridad y cifrado:** Redis puede configurarse con autenticación, ACLs y cifrado TLS, cumpliendo con el requisito de proteger datos en tránsito y en reposo.

Desventajas:

- **Datos volátiles:** Como Redis guarda todo en memoria, los datos pueden perderse si el servicio se reinicia sin mecanismos de persistencia.
- **Mayor consumo de recursos:** Al mantener los datos en memoria, Redis requiere más recursos de hardware en comparación con bases que trabajan sobre disco. Esto puede ser costoso si el volumen de datos temporales crece demasiado.

- Modelo de datos simple: Redis no está pensado para consultas complejas o relaciones entre datos.
- Persistencia limitada: Aunque Redis permite guardar snapshots o logs de operaciones, su persistencia no es tan robusta como la de un RDBMS. Esto puede ser un problema si se necesita auditar datos a largo plazo.

Trade-offs:

Redis prioriza la velocidad y la inmediatez por encima de la persistencia. Esto significa que los datos no quedan almacenados de forma permanente, pero en este caso no representa un inconveniente, ya que la información que maneja el subsistema de Privacidad y Seguridad (como sesiones y claves efímeras) está pensada para existir solo por un tiempo limitado.

También se sacrifica algo de complejidad en las consultas a cambio de simplicidad y rapidez. Redis funciona muy bien para operaciones puntuales, como validar un token o consultar una preferencia, pero no está diseñado para realizar búsquedas avanzadas o agregaciones de datos. En este contexto, eso no resulta problemático, porque los accesos son directos y no requieren análisis complejo.

Por último, mantener los datos en memoria implica un mayor consumo de recursos, especialmente de RAM. Sin embargo, este costo se considera razonable frente al beneficio que ofrece en términos de velocidad y seguridad en tiempo real, dos aspectos fundamentales para el correcto funcionamiento de este subsistema. Este costo se justifica por la necesidad de velocidad y seguridad en tiempo real.

Gestión de usuarios y perfiles

Motor de base de datos

Para el subsistema Gestión de usuarios y perfiles se usará MongoDB, ya que este subsistema almacena información altamente variable, semi-estructurada y en constante evolución, lo cual encaja perfectamente con las características de las bases de datos documentales. MongoDB es una opción ideal porque permite manejar de forma flexible y eficiente grandes volúmenes de datos heterogéneos que varían según cada usuario, como progreso, rachas, logros, amigos y configuraciones de privacidad.

Requerimientos no funcionales

A continuación, enunciamos los requerimientos no funcionales que identificamos y en los que basamos la selección de usar MongoDB:

Escalabilidad: *“Dado el gran volumen de usuarios en Duolingo...”*

“Es imprescindible que la implementación de estas mejoras no afecte la escalabilidad del sistema...”

El sistema debe ser capaz de crecer horizontalmente para soportar millones de perfiles, grandes volúmenes de lecturas (cuando el usuario abre su perfil) y actualizaciones constantes (racha, progreso, logros). MongoDB resuelve este requerimiento mediante sharding nativo, réplicas y escalabilidad horizontal transparente.

Flexibilidad y evolución del modelo de datos: *“Es crucial que la plataforma permita la incorporación de nuevas funcionalidades en el perfil de usuario de manera sencilla...”*

El modelo de datos debe admitir cambios frecuentes sin necesidad de rediseñar toda la estructura. MongoDB, al ser schema-less, permite agregar nuevos campos o estructuras al perfil sin migraciones masivas, facilitando pruebas A/B y evolución del producto.

Rendimiento / baja latencia: *“Su modelo freemium, accesible desde dispositivos móviles y web genera una enorme cantidad de datos provenientes del comportamiento de los usuarios...”*

“...es imprescindible que la implementación de estas mejoras no afecte la escalabilidad del sistema ni la fluidez de la experiencia de aprendizaje.”

Esto sugiere un requerimiento de alta velocidad tanto en lecturas como en escrituras, ya que el perfil se actualiza constantemente. MongoDB permite operaciones rápidas gracias a su modelo orientado a documentos y a la indexación eficiente de campos como email, username o identificadores del usuario.

Disponibilidad y continuidad del servicio: *“...es imprescindible que la implementación de estas mejoras no afecte la fluidez de la experiencia de aprendizaje.”*

Esto implica que el sistema debe permanecer disponible incluso ante actualizaciones o aumentos de carga. MongoDB replica los datos entre nodos y permite failover automático, garantizando continuidad sin interrupciones.

Seguridad y privacidad: *“Desde su perfil, el usuario también podrá administrar la configuración de privacidad y seguridad.”*

“Es crucial que la plataforma permita la incorporación de nuevas funcionalidades en el perfil de usuario de manera sencilla, sin comprometer la seguridad y privacidad de los datos.”

El sistema gestiona información personal sensible (correo, amigos, preferencias de privacidad, datos de progreso). Esto corresponde a un requerimiento de fuertes mecanismos de autenticación, control de acceso y encriptación. MongoDB ofrece cifrado en tránsito y en reposo, así como control de acceso basado en roles.

Capacidad para manejar datos heterogéneos: *“Dentro del perfil de usuario, se almacenan datos clave como el nombre de usuario, la foto de perfil, el progreso en los cursos, la racha de aprendizaje, los logros obtenidos y la lista de amigos con los que compiten o colaboran.”*

Este enunciado revela que los datos del perfil son altamente variados y en muchos casos anidados. MongoDB permite almacenar estas estructuras complejas directamente como documentos JSON, evitando tablas relacionales rígidas y múltiples joins.

Integración con arquitectura de microservicios: *“Cada subsistema maneja su propia base de datos, permitiendo elegir la tecnología que mejor se ajuste...”*

Esto exige una base de datos que funcione bien como backend independiente, con despliegue y escalado aislado. MongoDB se adapta perfectamente a esta arquitectura gracias a su independencia por servicio y su facilidad de despliegue distribuido.

Modelo de datos previsto

Colección usuario: Contiene la información principal de cada persona registrada en la plataforma. Contiene datos personales, configuraciones, y estado de suscripción.

Atributos principales del documento:

- `_id`: identificador único del usuario (UUID).
- `email`: correo electrónico de autenticación.
- `username`: nombre visible del usuario.
- `fotoPerfil`: URL de imagen.
- `fechaCreacion`: fecha de creación del perfil.

Subdocumento autenticación:

- `passwordHash`: hash de la contraseña.
- `tiene2FA`: indica si usa doble factor.
- `metodo2FA`: tipo de 2FA (email, app, sms).

Subdocumento configuracionPrivacidad:

- perfilVisible: si otros pueden ver su perfil.
- mostrarRacha: visibilidad de la racha.
- permitirMensajes: si permite mensajes de otros usuarios.
- compartirActividad: nivel de visibilidad (“amigos”, “todos”, “nadie”).

Subdocumento progresoGeneral:

- idiomaPrincipal: idioma principal de aprendizaje.
- nivelesCompletados: cantidad de niveles completados.
- xpTotal: experiencia total acumulada.

Array: cursos

Un usuario puede estar inscrito en varios cursos (uno por idioma). Cada elemento representa un curso activo:

CursoUsuario

- idioma: Idioma del curso.
- xpAcumulado: XP obtenida exclusivamente en ese curso.
- unidadesCompletadas: Cantidad de unidades finalizadas.
- fechaInscripcion: Fecha en la que comenzó el curso.
- unidadActual: Unidad donde se encuentra actualmente.
- nivelActual: Nivel actual dentro de la unidad.
- unidades: Lista de todas las unidades del curso (tipo UnidadCurso).

UnidadCurso

- numeroUnidad: Número de la unidad.
- nombre: Nombre identificador.
- nivelesTotales: Niveles que contiene la unidad.
- nivelesCompletados: Progreso real del usuario en esa unidad.
- completada: Indica si la unidad está finalizada.

Array logros: Cada elemento representa un logro obtenido por el usuario:

- idLogro: referencia a logro del catálogo.
- nombre: nombre del logro.
- fechaObtencion: fecha en que lo obtuvo.

Array amigos: Lista de amigos:

- idUsuario: ID del amigo.
- username: nombre del amigo.

Subdocumento suscripcion:

- tipo: “free” o “plus”.
- fechaInicio: inicio de la suscripción.
- fechaFin: vencimiento.
- autoRenovacion: indica si se renueva automáticamente.

Subdocumento preferencias:

- idiomasInteres: lista de idiomas que desea aprender.
- notificaciones:
 - email: si recibe notificaciones por mail.

- push: si recibe notificaciones push.
- recordatoriosDiarios: si recibe recordatorios diarios.

Colección actividad_usuario: Se utiliza para almacenar registros de actividad masivos, manteniéndolos fuera del documento de usuario por volumen.

Atributos:

- _id: identificador del documento de actividad.
- idUsuario: referencia al usuario.
- fecha: día al que corresponden las acciones.
- acciones: lista de acciones registradas durante el día:
 - tipo: tipo de acción (inicio_sesion, leccion_completada, etc.).
 - timestamp: momento exacto de la acción.
 - idLeccion: (opcional) para acciones asociadas al aprendizaje.
 - xpGanado: (opcional) experiencia obtenida.

Colección logros_definicion: Catálogo de logros de la plataforma, para referencia rápida por parte de los perfiles de usuario.

Atributos:

- _id: identificador del logro.
- nombre: nombre del logro.
- descripcion: detalle del logro.
- xpRecompensa: XP que otorga.

Colección estadistica_usuario: La colección estadisticas_usuario almacena estadísticas diarias detalladas del progreso de cada usuario.

Se usa para registrar hábitos, actividad diaria y métricas avanzadas del aprendizaje, permitiendo ofrecer funcionalidades como:

- Reportes históricos
- Seguimiento granular del aprendizaje por idioma
- Funciones exclusivas de Duolingo Plus (estadísticas avanzadas)
- Cada documento corresponde a un usuario en un día específico.

Métricas generales del día

- xpTotalDia: cantidad total de experiencia obtenida durante ese día (sumando todos los cursos).
- tiempoEstudioMinutos: Tiempo total de estudio registrado ese día, expresado en minutos.
- leccionesCompletadas: número total de lecciones completadas durante el día.
- unidadMasTrabajada: nombre o identificador de la unidad en la que más avanzó el usuario ese día.

Subdocumento: DetallePorCurso

- idioma: Idioma del curso (ej. "Inglés", "Portugués").
- xp: Experiencia obtenida en ese idioma durante el día.
- lecciones: Cantidad de lecciones completadas en ese curso.

Principales patrones de acceso identificados

Para este subsistema, los principales patrones de acceso están relacionados con la carga frecuente del perfil, las actualizaciones atómicas del progreso y la lectura eficiente de datos personales y de configuración.

Lecturas frecuentes del perfil del usuario

La plataforma carga el perfil del usuario constantemente: al iniciar sesión, al abrir la app, al cambiar de curso o al mostrar rankings. Estas consultas recuperan información como el nombre, avatar, idioma principal, racha, XP total y configuraciones de privacidad. Por ejemplo, cada vez que el usuario vuelve a la pantalla principal, la aplicación consulta su estado general para mostrar su avance y estadísticas.

Actualizaciones atómicas y frecuentes del progreso

El progreso del usuario se modifica decenas de veces por sesión: XP ganado, incremento de racha, avance de nivel, última actividad, entre otros. Estas operaciones realizan escrituras parciales sobre pocos atributos del documento, usando incrementos atómicos para asegurar que los valores no se corrompan incluso si el usuario usa varios dispositivos. Por ejemplo, al resolver un ejercicio se ejecuta un incremento en `progresoGeneral.xpTotal` y una actualización en la fecha de actividad.

Lecturas indexadas para autenticación y seguridad

El inicio de sesión depende de consultas rápidas por campos únicos como email o `nombreDeUsuario`. Estas operaciones requieren índices adecuados porque se ejecutan millones de veces por día y deben tener baja latencia. Por ejemplo, al ingresar a la plataforma se busca el usuario por correo electrónico para validar la contraseña y cargar sus datos.

Consultas parciales optimizadas para rankings y funciones sociales

Los rankings, listas de amigos y comparativas de rendimiento requieren lecturas filtradas que solo obtienen parte del documento: avatar, nombre, XP total, nivel o racha. MongoDB facilita este patrón usando proyecciones para enviar solo los campos necesarios y evitar transferencias de datos innecesarias. Por ejemplo, para mostrar la tabla de posiciones únicamente se leen los campos `nombre`, `avatar` y `progresoGeneral.xpTotal`.

Actualizaciones ocasionales del perfil y configuraciones

Cambios como la foto, el nombre, preferencias de privacidad o idioma principal ocurren con poca frecuencia, pero requieren modificaciones seguras sobre secciones específicas del documento del usuario. Por ejemplo, si el usuario modifica su configuración de privacidad, se actualiza únicamente el subdocumento `configuracionPrivacidad`.

Verificación diaria y actualización de la racha

Una operación clave del subsistema es verificar al inicio de cada día si el usuario mantiene, reinicia o pierde su racha de aprendizaje.

Este patrón combina lectura del campo `fechaUltimaActividad` y una posible actualización de `rachaActual`.

Por ejemplo, si pasaron más de 24 horas sin actividad, la racha se reinicia automáticamente.

Validaciones de seguridad antes de cambios sensibles

Antes de permitir un cambio de contraseña, correo o eliminación de cuenta, el sistema realiza verificaciones adicionales de identidad y estado del usuario.

Por ejemplo, antes de actualizar el email se verifica si ya está registrado por otro usuario.

Principales ventajas, desventajas y trade-offs

Ventajas:

Flexibilidad del esquema: MongoDB permite almacenar perfiles muy heterogéneos (racha, progreso, amigos, privacidad, XP, logros) sin necesidad de redefinir estructuras cada vez que Duolingo incorpora nuevas funciones. Esto es especialmente útil para una plataforma que evoluciona continuamente.

Alto rendimiento para lecturas frecuentes: El perfil del usuario se consulta todo el tiempo. MongoDB destaca en consultas rápidas basadas en documentos gracias a la indexación y a su modelo que evita joins complejos.

Escrituras atómicas parciales (ideal para progreso y rachas): Operaciones como sumar XP, actualizar la racha o registrar la última actividad se pueden realizar con operaciones atómicas (`$inc`, `$set`) sin afectar todo el documento.

Escalabilidad horizontal: MongoDB escala fácilmente mediante sharding, lo cual es crucial para millones de usuarios con perfiles que crecen en tamaño y actividad.

Integración natural con microservicios: Cada usuario es un documento independiente. Esta estructura encaja perfectamente con microservicios que necesitan límites claros y autonomía por base de datos.

Desventajas

Consistencia eventual en operaciones distribuidas: En escenarios donde la información del usuario es accedida o actualizada desde múltiples regiones, MongoDB puede aplicar consistencia eventual, lo cual puede generar pequeñas desincronizaciones temporales (por ejemplo, un delay en la sincronización de XP entre dispositivos).

Documentos que pueden crecer demasiado: Los perfiles de usuarios intensivos pueden convertirse en documentos grandes: listas de amigos, logros, configuraciones, estadísticas. Documentos demasiado grandes afectan rendimiento y costos.

No es ideal para relaciones complejas: Aunque MongoDB soporta referencias, no es óptimo para consultas que deben relacionar muchos datos entre distintas colecciones. El subsistema de usuarios tiene pocas relaciones, pero si crecieran, podría ser una limitación.

Falta de transacciones complejas entre colecciones: MongoDB sí soporta transacciones ACID, pero no están pensadas para ser usadas de forma intensiva ni con múltiples colecciones, a diferencia de una base relacional. Esto puede ser una limitación para cambios de privacidad o integraciones inter-servicios que afecten varios subsistemas.

Trade-offs

Se sacrifica consistencia estricta a cambio de rendimiento y escalabilidad: MongoDB prioriza velocidad y capacidad de manejar millones de usuarios. A cambio, en operaciones distribuidas la consistencia puede no ser inmediata. Para un perfil de usuario esto es aceptable: no se requiere consistencia bancaria.

Se elige flexibilidad sobre rigidez relacional: Duolingo evoluciona constantemente (nuevos logros, métricas de progreso, funciones). MongoDB evita migraciones costosas, pero pierde el fuerte control relacional.

Se almacenan documentos grandes para evitar joins: Este modelo mejora el rendimiento de lectura, pero aumenta el tamaño de cada documento y requiere un diseño cuidadoso para evitar crecimientos excesivos.

Se renuncia a consultas relacionales complejas: A cambio de tener consultas simples y rápidas que obtienen toda la información del perfil en un solo lookup. Es perfecto para lectura frecuente, pero limitado para análisis profundos o relaciones complejas.

Sistemas de lecciones y ejercicios

Motor de base de datos

Para el subsistema Sistemas de lecciones y ejercicios se usará PostgreSQL como base de datos relacional. Se busca un alto volumen de usuarios y la necesidad de actualizar y versionar cursos sin afectar la continuidad del aprendizaje. Su soporte ofrece transacciones ACID, por lo que las actualizaciones de contenido y el registro del progreso de los usuarios quedan siempre consistentes. Su modelo relacional permite guardar de forma clara la jerarquía *idioma–curso–unidad–lección–ejercicio*, y con sus índices y réplicas de lectura puede responder a muchas consultas al mismo tiempo. Además, el tipo JSONB permite guardar metadatos y configuraciones adicionales de los ejercicios de forma flexible, sin perder la estructura del modelo relacional.

Requerimientos no funcionales

A continuación, enunciamos los requerimientos no funcionales que identificamos y en los que basamos la selección de usar PostgreSQL:

Escalabilidad: *“Dado el alto volumen de usuarios y la diversidad de cursos disponibles...”*.

Esto indica que el sistema debe escalar para manejar muchas lecturas concurrentes y escrituras. PostgreSQL ayuda en esto con índices, optimizadores de consultas y la posibilidad de usar réplicas de lectura.

Disponibilidad y continuidad del servicio: *“asegurando que los cambios no afecten la continuidad del aprendizaje ni la estabilidad de la plataforma.”*

Requerimiento no funcional de alta disponibilidad y buenos mecanismos de recuperación. La base de datos debe poder seguir operando aun cuando se actualizan cursos o se realizan cambios de versión.

Consistencia de datos: *“la plataforma ofrece acceso secuencial, lo que significa que los usuarios deben completar ciertos requisitos antes de desbloquear nuevos niveles de aprendizaje.”*

Esto requiere una fuerte consistencia en el registro del progreso del usuario. Esto quiere decir que no se puede desbloquear una lección si la anterior no se marcó correctamente como completada. Las transacciones de PostgreSQL permiten garantizar que estas reglas se respeten incluso con muchos usuarios concurrentes.

Mantenibilidad y facilidad de actualización: *“Duolingo permite la edición frecuente y el versionado de sus cursos. Esto garantiza que los materiales de aprendizaje puedan ser ajustados y mejorados sin interrumpir la experiencia del usuario.”*

El sistema debe ser fácil de mantener y extender, admitiendo cambios en la estructura de cursos y ejercicios sin rediseñar todo. PostgreSQL facilita esto con migraciones estructuradas, vistas, funciones y tipos flexibles.

Rendimiento / baja latencia: implícitamente se pide un “*alto volumen de usuarios*” y la necesidad de “*continuidad del aprendizaje*”.

Buscamos que las consultas habituales como, carga de lecciones, ejercicios y progreso del usuario se resuelvan con baja latencia. Por eso se plantea índices adecuados y un diseño de consultas eficientes.

Seguridad y protección de datos de usuarios: Se asume que los usuarios deberán autenticarse para ingresar a la aplicación, ya que el sistema maneja información de progreso personal. Por ello se considera un requerimiento de seguridad para proteger estos datos, algo estándar en una plataforma educativa con muchos usuarios.

Modelo de datos previsto

El modelo de datos sigue la estructura que describe el texto: “*Cada idioma está compuesto por múltiples cursos... unidades... lecciones... ejercicios.*”

En PostgreSQL se plantea un esquema relacional donde cada nivel de la jerarquía tiene su propia tabla, enlazada por claves foráneas, y donde el progreso se guarda en tablas separadas para no mezclar contenido con datos de uso.

Tablas principales:

Idioma → almacena los idiomas disponibles en la plataforma.

Campos:

- *id_idioma*: es una pk, identificación del idioma
- *nombre*: nombre del idioma
- *código*: código corto del idioma

Curso → representa los cursos que pertenecen a un idioma.

Campos:

- *id_curso*: es una pk
- *id_idioma*: es una fk de la tabla idioma, a qué idioma pertenece el curso.
- *nombre*: nombre del curso
- *nivel*: es opcional, para indicar nivel o dificultad
- *id_version_actual*: es una fk de la tabla version_curso, hace referencia a la versión activa del curso.

Version_Curso → permite tener varias versiones históricas de un mismo curso.

Campos:

- *id_version*: es una pk
- *id_curso*: es una fk de la tabla curso
- *numero_version*: número o código de la versión
- *fecha_desde* / *fecha_hasta*: período de vigencia de la versión.
- *es_activa*: indica si la versión está disponible para nuevos usuarios.

Unidad → hace referencia a la unidad temática.

Campos:

- *id_unidad* : es una pk
- *id_version*: es una fk de la tabla *version_curso*, la versión concreta del curso.
- *nombre*: nombre de la unidad.
- *orden*: posición de la unidad dentro del curso.
- *id_unidad_requisito*: puede ser una fk de la tabla *unidad*. Nos permite establecer el “acceso secuencial”, por ejemplo, si el usuario termina la unidad 1 se puede desbloquear la unidad 2.

Leccion → lecciones específicas dentro de cada unidad.

Campos:

- *id_leccion*: es una pk
- *id_unidad*: es una fk de la tabla *unidad*
- *nombre*: nombre de la lección
- *orden*: posición dentro de la unidad
- *id_leccion_requisito*: puede ser una fk de la tabla *leccion*. Requisito de otra lección previa para desbloquear esta.

Ejercicio → representa los ejercicios que componen una lección. Aquí se aprovecha JSONB para configuraciones específicas.

Campos:

- *id_ejercicio*: es una pk
- *id_leccion*: es una fk de la tabla *leccion*.
- *tipo*: tipo de ejercicio (selección múltiple, completar huecos, traducción, etc.).
- *enunciado*: descripción de la consigna.
- *configuración* (JSONB): guarda opciones, respuestas correctas, ayudas, etc., de forma flexible.

Usuario → usuarios de la plataforma.

Campos:

- *id_usuario*: es una pk
- campos de autenticación y perfil

Progreso_Usuario_Leccion → el estado del usuario con respecto a cada lección completada.

Campos:

- *id_usuario*: es una pk , FK → USUARIO.
- *id_leccion*: es una pk, FK → LECCION.
- *estado*: pendiente, en_progreso, completada.
- *puntaje o porcentaje*: resultado obtenido.
- *fecha_ultimo_intento*: última vez que trabajó en esa lección.

Principales patrones de acceso identificados

Para este subsistema, los principales patrones de acceso tienen que ver con ...

Lectura jerárquica de contenido: los usuarios navegan por una estructura tipo idioma → curso → unidad → lección → ejercicio. Esto implica muchas lecturas encadenadas sobre tablas relacionadas, donde es importante que las consultas sean rápidas para no afectar la experiencia de aprendizaje.

Actualización frecuente de progreso: cada vez que el usuario avanza en una lección o ejercicio, se debe actualizar el progreso en la base de datos para mantener el “acceso secuencial” y poder determinar cual es el contenido que queda desbloqueado.

Lecturas para control de desbloqueo: el sistema consulta el progreso almacenado para verificar si el usuario cumplió los requisitos para desbloquear “nuevos niveles de aprendizaje”.

Actualización y versionado de contenido: relacionado a las operaciones de altas, bajas y modificaciones de cursos, unidades, lecciones y ejercicios, manteniendo múltiples versiones activas sin interrumpir la continuidad del aprendizaje.

Consultas agregadas: es un patrón razonable para poder monitorear la calidad de los cursos y mejorar el contenido.

Principales ventajas, desventajas y trade-offs

Ventajas:

- Representa la estructura jerárquica “idioma → curso → unidad → lección → ejercicio” usando modelo relacional e integridad referencial.
- Facilita aplicar reglas de acceso secuencial mediante restricciones y transacciones.
- Soporta edición frecuente y versionado de cursos manteniendo la continuidad del aprendizaje gracias a transacciones ACID y posibilidad de conservar versiones históricas.
- Es adecuado para el alto volumen de usuarios mencionado en el texto

Desventajas:

- Menor flexibilidad de esquema frente a cambios muy frecuentes, comparado con algunas bases NoSQL
- Requiere un diseño de datos cuidadoso, lo que aumenta el esfuerzo inicial y la complejidad de las migraciones.

Trade-offs:

- Se prioriza integridad, consistencia y estabilidad del aprendizaje por encima de la flexibilidad del esquema.
- La escalabilidad horizontal no es tan directa como en algunas NoSQL, a cambio de un modelo de datos robusto y coherente con la jerarquía, el acceso secuencial y el versionado de cursos.

Motor de recomendaciones

Motor de base de datos

Se usará una base de datos orientada a grafos como Neo4j ya que estas son ideales para usar en sistemas de recomendaciones en tiempo real, donde se pueden analizar datos correlacionados y donde el crecimiento de nodos es constante.

Neo4j se especializa en operar sobre relaciones, por lo que tiene sentido utilizarlo en este subsistema ya que debe detectar similitudes entre los patrones de aprendizaje de distintos usuarios, considerando su progreso, los errores o dificultades que pueda tener, así como similitudes entre usuarios.

Requerimientos no funcionales

A continuación, enunciamos los requerimientos no funcionales que identificamos y en los que basamos la selección de usar Neo4j:

Recomendaciones en tiempo real

Las recomendaciones que se realicen deben hacerse en el momento en el que el usuario esté usando la aplicación. De esta forma, la base de datos a usar no puede demorar mucho tiempo en generar las recomendaciones.

Flexibilidad

El motor de recomendaciones debe ser capaz de incorporar nuevos tipos de ejercicios, nuevas recomendaciones, etc. al tratarse de una aplicación que evoluciona constantemente. Particularmente, Neo4j tiene un modelo con alta flexibilidad llamado Relationships-first approach que es ideal para manejar esta necesidad, como se puede ver en su página web o en su documento sobre el uso de Neo4j en recomendaciones, en el Anexo.

Buen rendimiento con grandes cantidades de datos altamente relacionados

Las entidades como los usuarios, los ejercicios, el progreso, etc. se tienen como nodos y forman un grafo denso. Neo4j evita realizar JOINS como sí sucede en SQL y permite recorridos eficientes por relaciones múltiples.

Modelo de datos previsto

En el modelo de datos para el subsistema de Motor de recomendaciones se tendrán nodos para las entidades y para crear los grafos, habrán relaciones entre los nodos.

El subsistema tendrá los siguientes nodos:

- **Usuario:**

Es quien usa la aplicación y a quien se le mostrarán recomendaciones.

Tiene las siguientes propiedades:

- idUsuario
- username

- **Idioma:**
Tiene las siguientes propiedades:
 - idIdioma
 - nombre

- **Unidad:**
Bloque de contenido que tiene como propiedades:
 - idUnidad
 - nombre
 - posición
 - descripción
 - nivel

- **Habilidad:**
Competencia concreta de un idioma. Tiene como propiedades:
 - idHabilidad
 - nombre
 - descripción
 - categoría

- **Ejercicio:**
Actividad que debe resolver el usuario para avanzar en la unidad y tiene como propiedades:
 - idEjercicio
 - nombre
 - descripción
 - categoría

- **Error:**
Clasificación del tipo de error. Tiene las propiedades:
 - idError
 - descripción
 - categoría

Por su parte se tienen las relaciones entre cada nodo:

- **Usuario-ESTUDIA-Idioma**
Un usuario puede estudiar uno o varios idiomas. Tiene las propiedades:
 - nivel
 - fechaInicio
 - fechaUltimaActividad

- **Unidad-DEL_IDIOMA-Idioma**

- **Habilidad-DEL_IDIOMA-Idioma**

- **Ejercicio-PERTENECE_A-Unidad**
Un ejercicio pertenece a una unidad y tiene como propiedad:

- Posición
- **Ejercicio-REFUERZA-Habilidad**
Indica la habilidad que refuerza cierto ejercicio.
- **Usuario-REALIZA-Ejercicio**
Indica cada intento de un usuario sobre un ejercicio. Tiene como propiedad:
 - fechaRealizado
 - resultado
 - tiempo
 - intentos
- **Usuario-FALLA_EN-Habilidad**
Muestra parte del desempeño del usuario sobre una habilidad. Cuenta con la propiedad:
 - vecesFalladas
- **Usuario-SIMILAR_A-Usuario**
De esta forma se pueden detectar patrones similares entre los usuarios.
- **Unidad-PRECEDE_A-Unidad**
Las unidades de un mismo idioma tienen un orden de dictado.

Principales patrones de acceso identificados

En el motor de recomendaciones se identifican los siguientes patrones de acceso, en base a los tipos de análisis que Neo4j tiene como característicos para recomendaciones:

1. Recomendación basada en las dificultades del usuario

El subsistema de motor de recomendaciones debe obtener las habilidades en las que el usuario presenta mayores dificultades y luego buscar ejercicios que permitan reforzarlas

Este patrón recorre el grafo de la siguiente forma:

- Obtener las habilidades donde el usuario presenta fallas:
 - (:Usuario)-[:FALLA_EN]->(:Habilidad)
- Obtener los ejercicios que refuercen estas habilidades:
 - (:Ejercicio)-[:REFUERZA]->(:Habilidad)
- Filtrar los ejercicios según el idioma que el usuario está estudiando:
 - (:Usuario)-[:ESTUDIA]->(:Idioma)
 - (:Unidad)-[:DEL_IDIOMA]->(:Idioma)
 - (:Ejercicio)-[:PERTENECE_A]->(:Unidad)

La idea es poder recorrer el grafo para identificar datos importantes basándose en los errores y dificultades del usuario, lo más rápido posible

2. Recomendación basada en la similitud de usuarios

El subsistema también tiene que considerar los comportamientos similares entre los usuarios con el fin de recomendar ejercicios que hayan sido útiles para estudiantes con perfiles similares. El recorrido del grafo sigue este sería:

- Identificar usuarios similares con:
 - (:Usuario)-[:SIMILAR_A]->(:Usuario)
- Obtener ejercicios que estos usuarios realizaron:
 - (:Usuario)-[:REALIZA]->(:Ejercicio)
- Filtrar esos ejercicios por idioma usando las relaciones ESTUDIA y DEL_IDIOMA.

3. Recomendación basada en el contenido y la estructura del curso de cierto idioma

Las recomendaciones también pueden basarse en el curso, reforzando contenidos ya dados o anticipar los contenidos que se darán en el futuro. El recorrido podría ser:

- Obtener ejercicios pertenecientes a una unidad:
 - (:Ejercicio)-[:PERTENECE_A]->(:Unidad)
- Obtener la estructura de avance entre unidades:
 - (:Unidad)-[:PRECEDE_A]->(:Unidad)
- Detectar las habilidades asociadas a los ejercicios mediante:
 - (:Ejercicio)-[:REFUERZA]->(:Habilidad)
- Filtrar todo el contenido por idioma con:
 - (:Unidad)-[:DEL_IDIOMA]->(:Idioma)
 - (:Habilidad)-[:DEL_IDIOMA]->(:Idioma)

La idea es poder recomendar ejercicios adecuados al nivel del usuario de cierto idioma.

4. Recomendación según el idioma estudiado por el usuario

Como los usuarios pueden estudiar más de un idioma a la vez, los patrones de acceso tienen que realizarse dentro del subgrafo correspondiente al idioma particular que estudia. Se toma en cuenta:

- Idioma que estudia el usuario:
 - (:Usuario)-[:ESTUDIA]->(:Idioma)
- Unidades y habilidades asociadas a ese idioma:
 - (:Unidad)-[:DEL_IDIOMA]->(:Idioma)
 - (:Habilidad)-[:DEL_IDIOMA]->(:Idioma)
- Ejercicios pertenecientes a esas unidades:
 - (:Ejercicio)-[:PERTENECE_A]->(:Unidad)

5. Consultas según diversos criterios

El motor puede tener que combinar los criterios mencionados anteriormente para hacer recomendaciones más precisas.

Principales ventajas, desventajas y trade-offs

Ventajas:

- Tiene buen rendimiento en grafos densos. Al tener varios nodos para usuarios, ejercicios, habilidades, idiomas, unidades, etc., se genera un grafo muy interconectado. Según la documentación sobre recomendaciones de Neo4j que se puede ver en el anexo, está optimizado para hacer recorridos rápidos sobre estructuras de grafos densos.
- Es flexible y extensible. Como se mencionó previamente, Neo4j tiene un enfoque Relationship-First, que permite agregar entidades nuevas, tipos de ejercicios u otros criterios de recomendación sin cambios costosos del esquema. Esto va muy bien con Duolingo, porque es una aplicación que está constantemente evolucionando.
- Tiene varios enfoques de recomendación. El framework de Neo4j usa “hybrid scoring” que permite el uso simultáneo de varios métodos de recomendaciones para seleccionar cual es mejor usar en cada caso. Se tienen los siguientes tipos de scoring, donde se asigna un puntaje numérico con el fin de ordenarlos y elegir el mejor en cada caso:
 - Collaborative scoring. Estos métodos se basan en usuarios o productos similares.
 - Content scoring. Estos métodos se basan en historias de usuario y perfiles.
 - Rules-based scoring. Estos métodos se basan en reglas y criterios predefinidos.
 - Strategy scoring. Estos métodos están basados en promociones, márgenes, inventario y otros datos administrativos.

Desventajas:

- Tiene una curva de aprendizaje, donde lleva tiempo realmente entender cómo modelar con grafos, realizar consultas y llegar a conocer los algoritmos necesarios para aplicarlos correctamente en este motor de recomendaciones en particular.
- Requiere que se tenga infraestructura extra. Al ser el único subsistema para el cual se utilizará una base de datos orientada a grafos, esto implica que se introduce una tecnología nueva al sistema.

Trade-offs:

- Se prioriza la flexibilidad para evolucionar el modelo y poder extenderlo pero a cambio se añade un componente con cierta complejidad operacional que debe tener mantenimiento.
- Una virtud del uso de una base de datos orientada a grafos es que tiene muy buen rendimiento sobre grafos muy interconectados, con bastantes nodos y relaciones, pero esto a su vez implica un mayor costo de almacenamiento y mantenimiento.

Foros y comunidad

Motor de base de datos

Para este subsistema se utilizará Cassandra que es una base de datos NoSQL distribuida, orientada a columnas. Esta decisión se basa en varias características del requerimiento:

- se esperan “*espacios de discusión*”, “*lectura intensiva y escritura frecuente*” y una comunidad que “*crece*”, lo que implica un volumen muy alto de mensajes y lecturas concurrentes.
- Además, las discusiones se organizan en “*hilos con mensajes ordenados cronológicamente*” con un patrón de partición y clustering por tiempo.
- También se menciona la necesidad de “*mantener la accesibilidad y escalabilidad del sistema*” a medida que la comunidad crece.

Cassandra está optimizado para escritura y lectura rápidas en escenarios con muchos usuarios generando contenido al mismo tiempo. Su modelo de datos permite representar de forma eficiente hilos de mensajes ordenados por fechas, utilizando una clave de partición por hilo y una clave de clustering por timestamp. Además, su arquitectura distribuida facilita acompañar el crecimiento de la comunidad sin rediseñar el sistema, y su enfoque en alta disponibilidad resulta clave para que los foros se mantengan accesibles y las conversaciones puedan seguir desarrollándose en cualquier momento.

Requerimientos no funcionales

A continuación, enunciamos los requerimientos no funcionales que identificamos y en los que basamos la selección de usar Cassandra:

Baja latencia en lectura y escritura:

El subsistema está diseñado para “*incentivar la lectura intensiva y la escritura frecuente*”. Esto implica que el sistema debe poder insertar mensajes nuevos y leer hilos completos rápidamente, incluso con muchos usuarios conectados al mismo tiempo.

Escalabilidad y crecimiento de la comunidad:

“*A medida que la comunidad crece, es fundamental mantener la accesibilidad y escalabilidad del sistema para que las discusiones sean eficientes y beneficiosas para todos los usuarios.*”.

Cassandra está diseñada para escalar horizontalmente, repartiendo los datos entre muchos nodos y poder mantener un rendimiento estable a medida que aumentan el volumen de datos y los usuarios concurrentes.

Alta disponibilidad y experiencia continua:

“*Para garantizar una experiencia enriquecedora*”, “*mensajes ordenados cronológicamente, lo que facilita el seguimiento de las conversaciones*”.

En este subsistema de foros la experiencia de aprendizaje y de comunidad es importante. Si estos espacios no están disponibles, se pierde una parte importante del valor de la

plataforma. Cassandra nos da la capacidad de seguir atendiendo lecturas y escrituras incluso ante fallos parciales.

Consistencia eventual aceptable:

Se consideró que en un foro, pequeños retrasos en la propagación de un mensaje o en el conteo de respuestas no afectan la corrección del aprendizaje central. Por lo tanto, se puede aceptar consistencia eventual, priorizando rendimiento y disponibilidad. Que un mensaje tarde un poco en verse en todos los nodos, sin romper el uso normal del foro.

Soporte para moderación y filtros de contenido:

“ el sistema puede incorporar moderación automática y filtros de contenido “.

Esto implica que se debe marcar mensajes para revisión y registrar reportes o estado de moderación.

Cassandra puede almacenar estos datos asociados a cada mensaje o usuario.

Modelo de datos previsto

En Cassandra el modelo de datos se diseña en función de las consultas que se van a realizar con más frecuencia. Para un foro, las operaciones típicas son:

- Listar los mensajes de un hilo en orden cronológico.
- Listar los hilos de un área/tema ordenados por actividad reciente.
- Ver los mensajes que ha escrito un usuario.

Esto nos lleva a un modelo basado en tablas:

Tabla principal:

posts_by_thread → almacena los mensajes de cada hilo.

- Clave de partición:
thread_id → Todos los mensajes de un mismo hilo se guardan en la misma partición.
- Claves de clustering:
created_at → orden cronológico dentro del hilo.
post_id → identificador del mensaje, para diferenciar si dos tienen el mismo timestamp.

Columnas de datos:

- *author_id*
- *content* → texto del mensaje
- *language_code / course_id* → para relacionar el hilo con un idioma/curso, si aplica
- *parent_post_id* → si es una respuesta a otro mensaje
- *likes_count*
- *flags / moderation_status* → para moderación y filtros

Con este modelo, leer un hilo completo en orden cronológico es más simple.

Tabla:

threads_by_forum → para listar los hilos de un foro/tema.

- Clave de partición:
forum_id → puede representar el foro de un idioma, un curso o una categoría temática.
- Claves de clustering:
last_activity_at → última actividad en el hilo; permite ordenar hilos por actividad reciente.
thread_id

Columnas de datos:

- *title* → título del hilo
- *created_by* → usuario que creó el hilo
- *created_at*
- *reply_count* / *views_count*

Esto permite listar rápidamente los hilos de un foro, con los más activos primero.

Tabla:

posts_by_user → mostrar la actividad de un usuario

- Clave de partición:
user_id
- Claves de clustering:
created_at
post_id
thread_id

Principales patrones de acceso identificados

Para este subsistema de foro y comunidad, los principales patrones de acceso tienen que ver con el modelo orientado a escritura intensiva de mensajes nuevos y lectura por clave que ofrece Cassandra, y explican por qué no se busca un esquema relacional con joins, sino tablas pre-pensadas para cada tipo de consulta.

Lectura de hilos en orden cronológico:

“ *Las discusiones se organizan en hilos con mensajes ordenados cronológicamente* ”.

Patrón: *SELECT * FROM posts_by_thread WHERE thread_id = ?* ordenando por *created_at*.

Escritura frecuente de mensajes:

Se habla de “*escritura frecuente*” y de usuarios que “*hacen preguntas*”, responden, comparten experiencias, etc.

Patrón: INSERT en *posts_by_thread* cada vez que se agrega un mensaje, y potencialmente actualización de *threads_by_forum* para refrescar *last_activity_at* y *reply_count*.

Lectura de listas de hilos:

Para navegar la comunidad, los usuarios verán listas de hilos por foro o tema, con los más activos arriba.

Patrón: *SELECT ... FROM threads_by_forum WHERE forum_id = ?* ordenado por *last_activity_at* (clustering key).

Consulta de actividad de un usuario:

“ los aprendices pueden motivarse entre sí, compartir experiencias y resolver dudas en conjunto ” Para poder ver lo que escribió un usuario:

Patrón: *SELECT ... FROM posts_by_user WHERE user_id = ?*

Moderación y filtros de contenido:

“ el sistema puede incorporar moderación automática y filtros de contenido ”.

Patrón: actualizar *moderation_status* o *flags* en *posts_by_thread*

Principales ventajas, desventajas y trade-offs

Ventajas:

- Cassandra permite manejar un volumen muy alto de hilos y mensajes, lo que se ajusta a una comunidad global que crece constantemente.
- Lectura y escritura rápidas para patrones simples (mensajes por hilo, hilos por foro), lo que ayuda a cumplir con la idea de *“lectura intensiva y escritura frecuente”* sin perjudicar la experiencia.
- Para poder manejar los datos ordenados por tiempo los hilos con mensajes cronológicos encajan bien con el patrón de partición por *thread_id* y clustering por *created_at*.
- La alta disponibilidad contribuye a mantener los foros accesibles, reforzando el rol de la comunidad como espacio de apoyo constante.
- Mantenemos una consistencia eventual, pequeños retrasos en la propagación de mensajes o contadores no rompen la funcionalidad del foro.

Desventajas:

- Con el modelo orientado a consultas el diseño de tablas depende fuertemente de los patrones de acceso. Cambios en las formas de consultar (por ejemplo, nuevas vistas o filtros complejos) pueden requerir nuevas tablas y mayor duplicación de datos.
- Pocas capacidades para consultas ad-hoc y joins: Cassandra no es adecuada para hacer análisis complejos o combinaciones ricas de datos. Para eso suele ser necesario un sistema de analítica aparte.
- Gestión de datos duplicados: al estar denormalizado, el mismo mensaje puede estar en varias tablas (por hilo, por usuario, por foro), lo que complica ciertas actualizaciones (por ejemplo, moderación) y consume más almacenamiento.

Trade-offs:

- Se elige un modelo denormalizado y orientado a consultas, que simplifica y acelera las operaciones críticas para el usuario (leer hilos, publicar mensajes), a cambio de mayor complejidad en la actualización y en la evolución del esquema.
- Se prioriza la escalabilidad y disponibilidad de la comunidad (que siempre haya foros activos y accesibles) por encima de la consistencia fuerte inmediata en todos los nodos, aceptando consistencia eventual como un compromiso razonable.
- Se asume que las necesidades de búsqueda avanzada o analítica profunda (por ejemplo, full-text search en posts, estadísticas complejas) se resolverán con herramientas adicionales (motores de búsqueda), mientras que Cassandra se centra en el almacenamiento operativo de los hilos y mensajes.

Sistemas de notificaciones

Motor de base de datos

Para este subsistema se usará Cassandra que es una base de datos NoSQL distribuida, orientada a columnas. Esta elección se consideró porque en la descripción de los requerimientos se establece que:

- Las notificaciones deben de tener un tiempo de respuesta optimizado.
- Llegar en el momento adecuado
- manejar un tiempo de vida (TTL) adecuado para cada tipo de notificación y puedes descartarlas cuando no son útiles.

Cassandra se especializa en almacenar y consultar grandes volúmenes de datos distribuidos, con escrituras muy frecuentes, alta disponibilidad y soporte nativo de TTL, lo que la hace especialmente adecuada para subsistemas de notificaciones y eventos a gran escala.

Requerimientos no funcionales

A continuación, enunciamos los requerimientos no funcionales que identificamos y en los que basamos la selección de usar Cassandra:

Baja latencia: *“Estas notificaciones tienen un tiempo de respuesta optimizado para garantizar que los mensajes lleguen en el momento adecuado.”*

Esto requiere que el sistema va a tener que poder leer y escribir notificaciones muy rápidamente, incluso bajo cargas muy altas. Cassandra permite esto porque está diseñada para ofrecer tiempos de respuesta constantes y bajos en operaciones simples de lectura y escritura.

Gestión de TTL y relevancia temporal: *“ el sistema maneja un tiempo de vida (TTL) adecuado para cada tipo de notificación. ” y “se descartan cuando dejan de ser útiles”.*

Con el TTL nativo de Cassandra le podemos asignar un tiempo de vida al insertar la notificación y el propio motor se encarga de eliminarla automáticamente cuando expira. Esto se alinea directamente con la necesidad de descartar notificaciones que ya no son útiles.

Escalabilidad masiva y retención de estudiantes: *“ El sistema de recordatorios juega un papel clave en la retención de los estudiantes, enviando alertas sobre la racha de aprendizaje, lecciones pendientes o la necesidad de reforzar ciertos temas. ”*

Considerando que vamos a trabajar en un entorno con millones de usuarios, esto genera un volumen enorme de eventos de notificación. Para eso se requiere una base que pueda escalar horizontalmente sin perder rendimiento, y esa es una de las principales fortalezas de Cassandra.

Alta disponibilidad y tolerancia a fallos: *“ asegurando una experiencia fluida y personalizada dentro de la plataforma. ”*

Cassandra está pensado precisamente para entornos de alta disponibilidad, con replicación

de datos entre nodos y datacenters, de forma que el subsistema de notificaciones pueda seguir funcionando incluso ante fallos de algunos nodos.

Consistencia fuerte: perder una notificación puntual o recibirla dos veces no rompe el sistema de aprendizaje pero sí afecta la experiencia del usuario. Por eso, en este subsistema no es imprescindible la consistencia fuerte: podemos usar niveles de consistencia más livianos en Cassandra para priorizar rendimiento y disponibilidad, aceptando consistencia eventual como un trade-off razonable.

Modelo de datos previsto

En Cassandra el modelo de datos se diseña a partir de las consultas.

Para el subsistema de notificaciones interesa principalmente:

- Guardar notificaciones por usuario y por momento.
- Saber qué notificaciones están pendientes de enviar.
- Aplicar TTL para que desaparezcan automáticamente cuando dejan de ser relevantes.

Una posible tabla principal podría ser:

notifications_by_user

- Clave de partición:
user_id → Agrupa todas las notificaciones de un usuario en la misma partición.
- Claves de clustering:
scheduled_at → orden temporal dentro de la partición
notification_id → para diferenciar notificaciones con la misma hora
- Columnas de datos (ejemplo):
type : (recordatorio_racha, leccion_pendiente, logro, refuerzo_tema, etc.)
title, message → texto de la notificación
status → pendiente, enviada, leída, expirada
created_at
expires_at → fecha de caducidad lógica, además del TTL
payload → mapa o texto con información adicional: lección, unidad, racha, puntaje, etc.

En Cassandra se puede insertar cada notificación con un TTL definido (por ejemplo, acorde al tipo de notificación). De esta forma, aunque *expires_at* guarde la fecha de expiración lógica, el almacenamiento se libera automáticamente pasado ese tiempo, cumpliendo lo que pide el requerimiento.

Principales patrones de acceso identificados

Para este subsistema de notificación, los principales patrones de acceso tienen que ver con el modelo orientado a escritura rápida y lectura por clave/tiempo que ofrece Cassandra, y

explican por qué no se busca un esquema relacional con joins, sino tablas pre-pensadas para cada tipo de consulta.

Inserción de notificaciones cuando ocurren eventos relevantes:

Cuando se detectan situaciones como “lecciones pendientes” o un nuevo logro, el sistema genera una o varias notificaciones.

Patrón: muchas escrituras (*INSERT*) en *notifications_by_user* cada una con TTL.

Lectura por usuarios para mostrar notificaciones recientes:

Cuando el usuario abre la aplicación, se pueden leer sus notificaciones más recientes.

Patrón: *SELECT* sobre *notifications_by_user* filtrando por *user_id* y, opcionalmente, por rango de *scheduled_at* o *status*.

Procesamiento de notificaciones pendientes para envío:

Un proceso en background necesita encontrar las notificaciones que deben ser enviadas ahora o en los próximos minutos.

- Si se usa solo *notifications_by_user*, se puede recorrer por usuario.
- Si se usa también *notifications_by_time*, el proceso lee la partición correspondiente al “time_bucket” actual y obtiene todas las notificaciones con *scheduled_at* <= *ahora* y *status* = ‘pendiente’.

Expiración automática de notificaciones:

Gracias al TTL, Cassandra borra las notificaciones que ya no son útiles, cumpliendo con “se descartan cuando dejan de ser útiles, evitando saturar al usuario con información desactualizada”.

No es un patrón de acceso explícito de la aplicación, pero es un comportamiento clave del almacenamiento.

Actualización del estado de notificación:

Cuando una notificación se envía o el usuario la ve, el sistema puede actualizar el *status* a “enviada” o “leída”.

No se trata de un patrón de actualizaciones masivas, por lo que encaja bien con Cassandra, donde las escrituras (*INSERT/UPDATE*) están optimizadas.

Patrón: *UPDATE* de una fila específica (mismo *user_id*, *scheduled_at*, *notification_id*).

Principales ventajas, desventajas y trade-offs

Ventajas:

- Cassandra está diseñada para agregar nodos y repartir datos automáticamente, lo que permite manejar un enorme volumen de notificaciones y usuarios sin perder rendimiento, algo coherente con la idea de mantener “el compromiso y la motivación” de muchos estudiantes a la vez.
- El soporte de TTL a nivel de fila/columna satisface directamente el requisito de manejar un “*tiempo de vida (TTL) adecuado para cada tipo de notificación*” y “descartar” notificaciones cuando ya no son relevantes.

- La arquitectura distribuida de Cassandra prioriza tiempos de respuesta bajos y tolerancia a fallos, ayudando a cumplir con el “*tiempo de respuesta optimizado*” y la “*experiencia fluida*” que menciona el texto.
- Modelo de datos natural para eventos temporales. Particionar por usuario y ordenar por tiempo hace que consultas del tipo “últimas notificaciones de este usuario” o “notificaciones a enviar en esta ventana” sean muy eficientes.
- Consistencia ajustable : Para muchas notificaciones de recordatorio, consistencia eventual es aceptable y se puede priorizar el rendimiento.

Desventajas:

- El esquema se diseña en función de cómo se va a leer y escribir, no en términos de normalización. Esto puede resultar menos intuitivo y más rígido si se cambian mucho los patrones de acceso.
- No está pensada para hacer análisis complejos combinando notificaciones con otros datos (por ejemplo, progreso detallado de cursos); para eso harían falta otros sistemas de analítica o procesos de exportación.
- Menos soporte de transacciones multi-fila/multi-partición porque no ofrece el mismo nivel de transacciones que una base de datos relacional, lo cual complica si se quisieran operaciones muy fuertes entre muchas notificaciones o entre subsistemas.

Trade - offs:

- Se acepta consistencia eventual y la posibilidad de que una notificación se pierda o llegue dos veces, a cambio de conseguir muy alta escalabilidad y baja latencia, algo razonable para un subsistema donde la prioridad es “*llegar en el momento adecuado*” y mantener la motivación, no guardar un historial perfecto.
- Se adopta un modelo de datos des-normalizado, con posibles duplicaciones para poder optimizar las consultas típicas del sistema.

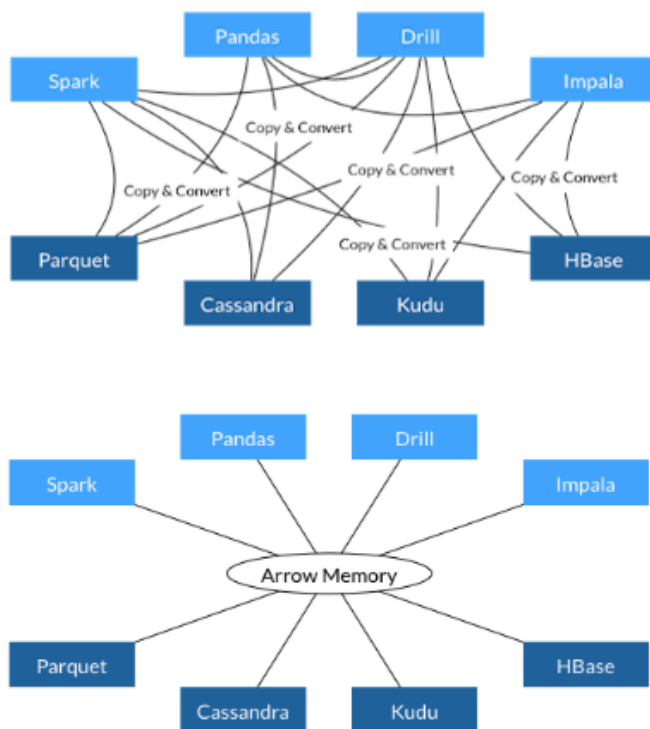
Requerimiento 2

Apache Arrow es un formato columnar en memoria, ideal para el análisis o lectura de datos y no para la escritura o modificación de estos, ya que no está pensado para el almacenamiento persistente de los mismos.

Por su parte, Apache Arrow también es independiente del lenguaje y cuenta con un sistema de tipos (ejemplo: tipos de datos anidados y definidos por el usuario) para que cumpla con las necesidades de sistemas de bases de datos analíticas.

Al ser un formato columnar, tiene como ventaja la rapidez y eficiencia con la que puede leer e iterar grandes cantidades de datos.

Apache Arrow permite la vectorización, al utilizar SIMD (Single Instruction, Multiple Data), siempre que los datos estén contiguos en memoria y que sean del mismo tipo de dato.



Cuando trabajas con datos (Pandas, Spark, bases de datos, etc.), cada herramienta usa su propio formato interno, y mover datos entre ellas exige serializar, deserializar y copiar en memoria. Apache Arrow estandariza el layout columnar en memoria, así que múltiples herramientas pueden compartir datos sin convertirlos (o con costo mínimo). Esto reduce copias y conversiones, permite reutilizar algoritmos entre lenguajes y acelera el análisis gracias a la lectura secuencial y la vectorización. Además, el proyecto ofrece bibliotecas oficiales (C++, Java, Python, R, Rust, JavaScript, C#, entre otras) y componentes para E/S remota y transferencia por red en formato Arrow.

Detalle técnico:

Tipos de datos

El archivo [Schema.fbs](#) define los tipos de datos integrados compatibles con el formato de columnas Arrow. Cada tipo de dato utiliza una disposición física bien definida.

Tipo	Parámetros de tipo	Disposición de la memoria física
Nulo		Nulo
Booleano		Primitivo de tamaño fijo
Int	- Ancho de bits - Signatura	Primitivo de tamaño fijo
Punto flotante	Precisión	Primitivo de tamaño fijo
Decimal	- Ancho de bits - Escala - Precisión	Primitivo de tamaño fijo
Fecha	Unidad	Primitivo de tamaño fijo
Tiempo	- Ancho de bits (es redundante porque cada unidad exige un solo ancho de bit) - Unidad	Primitivo de tamaño fijo
Marca de tiempo	- Unidad - Zona horaria	Primitivo de tamaño fijo
Intervalo	Unidad	Primitivo de tamaño fijo
Duración	Unidad	Primitivo de tamaño fijo
Binario de tamaño fijo	Ancho de bytes	Binario de tamaño fijo
Binario		Binario de tamaño variable con desplazamientos de 32 bits
UTF8		Binario de tamaño variable con desplazamientos de 32 bits
Binario grande		Binario de tamaño variable con desplazamientos de 64 bits
UTF8 grande		Binario de tamaño variable con desplazamientos de 64 bits

		bits
Vista binaria		Vista binaria de tamaño variable
Vista UTF8		Vista binaria de tamaño variable
Lista de tamaño fijo	- <i>Tipo de valor</i> - Tamaño de la lista	Lista de tamaño fijo
Lista	<i>Tipo de valor</i>	Lista de tamaño variable con desplazamiento de 32 bits
Lista grande	<i>Tipo de valor</i>	Lista de tamaño variable con desplazamiento de 64 bits
Vista de lista	<i>Tipo de valor</i>	Vista de lista de tamaño variable con desplazamientos y tamaños de 32 bits
Vista de lista grande	<i>Tipo de valor</i>	Vista de lista de tamaño variable con desplazamientos y tamaños de 32 bits
Estructura	<i>Hijos</i>	Estructura
Mapa	- <i>Hijos</i> - Ordenación de claves	Lista de estructuras de tamaño variable
Unión	- <i>Hijos</i> - Modo - Identificadores de tipo	
Diccionario	- <i>Tipo de índice (4)</i> - <i>Tipo de valor</i> - Orden	Diccionario codificado
Codificación de fin de ejecución	- <i>Tipo de fin de ejecución (5)</i> - <i>Tipo de valor</i>	Codificación de fin de ejecución

- Los parámetros de tipo en cursiva indican los tipos secundarios de un tipo de datos.
- (4) El tipo de índice de un tipo Diccionario sólo puede ser un tipo entero, preferiblemente con signo (ejemplo: int*) , con un ancho de 8 a 64 bits.
- El run end (el valor que marca la posición final de cada racha/run) debe ser un entero con signo de 16, 32 o 64 bits.

Representación física:

Arrow organiza los datos en columnas (“arrays”) almacenadas en buffers contiguos para favorecer el acceso secuencial y la ejecución vectorizada. Cada array se describe por su tipo lógico, una secuencia de buffers físicos, la longitud lógica (enteros con signo de 64 bits) y el recuento de nulos (también int64). En tipos anidados, cada array hijo replica la misma estructura, de modo que un array complejo siempre se puede entender como una composición de arrays más simples. La representación física se apoya en tres piezas clave: un bitmap de validez opcional (un bit por valor; 1 = válido, 0 = nulo), un buffer de datos que contiene los valores “crudos” y, cuando los valores tienen tamaño variable, un buffer de offsets con enteros de 32 o 64 bits que marcan la posición de inicio de cada valor y permiten derivar su longitud por diferencias consecutivas. La longitud y el recuento de nulos se almacenan como metadatos del array para facilitar validaciones y operaciones de alto nivel sin recorrer todos los elementos.

En los tipos primitivos de tamaño fijo (por ejemplo, int32, float64), cada elemento ocupa exactamente el mismo número de bytes y los valores se guardan densamente en un único buffer de datos; el bitmap de validez se incluye solo si hay nulos.

En los tipos binarios o de texto de tamaño variable, cada valor puede tener longitud distinta; por eso la disposición física combina el bitmap de validez (si aplica), un buffer de offsets que señala el comienzo de cada cadena o blob y un buffer de datos donde se concatenan los bytes reales. El mismo patrón de offsets se utiliza en las listas de tamaño variable: la diferencia es que, en lugar de apuntar a un buffer de datos “en bruto”, los offsets referencian un array hijo que contiene los elementos de la lista, lo que permite modelar colecciones heterogéneas y anidadas manteniendo la contigüidad columnar. Para listas de tamaño fijo, la implementación es aún más directa: no se necesitan offsets porque cada ranura ocupa exactamente N elementos del array hijo; la posición de cada sub-secuencia se calcula aritméticamente ($\text{índice} \times N$), con el bitmap de validez del padre controlando la nulidad a nivel de la lista.

Las estructuras (structs) se representan como varios arrays hijos, uno por campo, almacenados en paralelo y con la misma longitud lógica. El array padre puede tener su propio bitmap de validez, independiente del bitmap de cada hijo: la validez efectiva de un campo en una posición dada surge de combinar la validez del padre con la del hijo (típicamente, mediante un AND lógico) cuando se requiere una comprobación precisa. Las uniones permiten mezclar varios tipos en una sola columna manteniendo el procesamiento columnar. En la variante densa, además de los arrays hijos (uno por alternativa de tipo), se guarda un buffer de identificadores de tipo por elemento y un buffer de offsets que indica la posición del valor dentro del hijo correspondiente; en la variante dispersa no hay offsets y los hijos tienen la misma longitud que la unión, lo cual simplifica el direccionamiento a costa de ocupar más memoria. En ambos casos se preserva la propiedad fundamental de Arrow: los datos de cada componente siguen en buffers contiguos aptos para vectorización.

La codificación por diccionario se utiliza como técnica transversal de compresión lógica cuando hay muchos valores repetidos. El array de datos almacena índices enteros no negativos y el diccionario, almacenado como un array aparte de cualquier tipo, contiene los valores únicos. Esta separación permite que el array principal tenga la misma disposición

que un array de enteros primitivos (bitmap de validez opcional + buffer de índices) mientras que el diccionario, también columnar, puede reutilizarse y transmitirse por separado. En escenarios multilenguaje, aunque Arrow define longitudes y recuentos en 64 bits, es habitual imponer límites prácticos (por ejemplo, $2^{31}-1$) por compatibilidad con runtimes que no manejan bien enteros sin signo o muy anchos; colecciones mayores se particionan en fragmentos múltiples sin perder el modelo columnar.

Para sostener el rendimiento de memoria y CPU, los buffers se alinean y rellenan a múltiplos de 8/64 bytes. La alineación garantiza accesos alineados, reduce líneas de caché parcialmente utilizadas y facilita que los compiladores y las bibliotecas vectoriales (SIMD) generen rutas de ejecución predecibles y eficientes sin comprobaciones adicionales. Ese padding no tiene significado semántico: no codifica datos, solo asegura que los verdaderos contenidos comienzan en direcciones favorables para el hardware.

Implementación

Para la implementación se usará el lenguaje Python y Google Colab ya que Apache Arrow tiene soporte en Python con la librería pyarrow. El archivo .ipynb, junto con el dataset utilizado data.csv, se encuentran en la carpeta entregada comprimida .zip.

Para mostrar las características de compresión de Apache Arrow, se usó Apache Parquet ya que es este formato quien se encarga de guardar en disco en vez de en memoria. También se utilizó Feather, que es un formato construido sobre Apache Arrow, que permite serialización y deserialización de datos, usando los compresores que soporta (LZ4 y ZSTD).

Arrow no se centra en la compresión como Parquet o ORC. Su prioridad es la velocidad en memoria, por eso el layout columnar se mantiene sin comprimir para procesar más rápido. Pero cuando se serializan los datos (para guardarlos o enviarlos), se puede comprimir por bloque/buffer de forma transparente.

En el ecosistema Arrow se usan compresores estándar como Gzip, ZSTD (Zstandard) y Snappy. Esta compresión se aplica al serializar con Arrow IPC, Arrow Flight y el Arrow File Format (Feather), reduciendo el tamaño en tránsito o en disco sin cambiar la estructura lógica de los datos.

Si bien Apache Arrow no tiene una función en pyarrow para comprimir tablas en memoria, este ya optimiza para la eficiencia al ser un formato columnar y al usar codificaciones como por diccionario, para columnas con valores repetidos. De esta forma, Arrow logra cierta compresión de datos en memoria, representando de forma eficiente los datos.

Requerimiento 3

Para el subsistema de motor de recomendaciones se utiliza Neo4j, que implementa el modelo de property graph, donde las entidades del dominio son nodos con una etiqueta y propiedades, y las interacciones entre estos se modelan como relaciones dirigidas, que en caso de ser necesario, también tienen propiedades.

Este modelo cumple con lo que pide el subsistema, ya que debe trabajar con datos muy relacionados como usuarios, idiomas, unidades, ejercicios, habilidades, etc.

Modelado genérico:

Al igual que como se mencionó en el requerimiento 1, se tienen los siguientes nodos y relaciones:

- Nodos:
 - Usuario
Propiedades: idUsuario, username.
 - Idioma
Propiedades: idIdioma, nombre.
 - Unidad
Propiedades: idUnidad, nombre, posicion, descripcion, nivel.
 - Habilidad
Propiedades: idHabilidad, nombre, descripcion, categoria.
 - Ejercicio
Propiedades: idEjercicio, nombre, descripcion, categoria.
 - Error
Propiedades: idError, descripcion, categoria
- Relaciones:
 - (:Usuario)-[:ESTUDIA {nivel, fechaInicio, fechaUltimaActividad}] → (:Idioma)
 - (:Unidad)-[:DEL_IDIOMA] → (:Idioma)
 - (:Habilidad)-[:DEL_IDIOMA] → (:Idioma)
 - (:Ejercicio)-[:PERTENECE_A {posicion}] → (:Unidad)
 - (:Ejercicio)-[:REFUERZA] → (:Habilidad)
 - (:Usuario)-[:REALIZA {fechaRealizado, resultado, tiempo, intentos}] → (:Ejercicio)
 - (:Usuario)-[:FALLA_EN {vecesFalladas}] → (:Habilidad)
 - (:Usuario)-[:SIMILAR_A] → (:Usuario)
 - (:Unidad)-[:PRECEDE_A] → (:Unidad)

De esta forma se puede crear un grafo donde es posible recorrer las relaciones entre los usuarios, los idiomas, las habilidades y errores, entre otros, para luego realizar recomendaciones, cumpliendo con lo pedido por el subsistema.

Ejemplo:

Dado un ejemplo genérico donde un usuario estudia inglés, con una unidad, dos ejercicios y una habilidad asociada, los datos se verían de la siguiente forma:

- La usuaria Laura (Usuario) estudia Inglés (Idioma) en nivel A1.
- Una unidad Basico 1 (Unidad) del idioma inglés.
- Esta unidad tiene dos ejercicios (Ejercicio).
- Este ejercicio refuerza la habilidad Present simple (Habilidad)
- El usuario hace un ejercicio y falla.

En el lenguaje de consultas que usa Neo4j, Cypher, estos datos se podrían crear de la siguiente forma:

```
CREATE (u:Usuario {
  idUsuario: 'u1',
  username: 'Laura'
});
```

```
CREATE (i:Idioma {
  idIdioma: 'en',
  nombre: 'Inglés'
});
```

```
MATCH (u:Usuario {idUsuario: 'u1'}), (i:Idioma {idIdioma: 'en'})
CREATE (u)-[:ESTUDIA {
  nivel: 'A1',
  fechaInicio: date('2025-01-10'),
  fechaUltimaActividad: date('2025-09-25')
}]->(i);
```

```
CREATE (un:Unidad {
  idUnidad: 'u_basic1_en',
  nombre: 'Básico 1',
  posicion: 1,
  descripcion: 'Unidad introductoria de inglés',
  nivel: 'A1'
});
```

```
MATCH (un:Unidad {idUnidad: 'u_basic1_en'}), (i:Idioma {idIdioma: 'en'})
CREATE (un)-[:DEL_IDIOMA]->(i);
```

```
CREATE (h:Habilidad {
  idHabilidad: 'h_present_simple',
  nombre: 'Present simple',
  descripcion: 'Uso del present simple en una oración',
  categoria: 'gramática'
});
```

```
MATCH (h:Habilidad {idHabilidad: 'h_present_simple'}), (i:Idioma {idIdioma: 'en'})
CREATE (h)-[:DEL_IDIOMA]->(i);
```

```
CREATE (e1:Ejercicio {
  idEjercicio: 'e101',
```

```
    nombre: 'Crear oraciones present simple 1',
    descripcion: 'Crear oraciones negativas en present simple',
    categoria: 'crear'
  });
```

```
CREATE (e2:Ejercicio {
  idEjercicio: 'e102',
  nombre: 'Completar present simple 1',
  descripcion: 'Completar los verbos en present simple',
  categoria: 'completar'
});
```

```
MATCH (un:Unidad {idUnidad: 'u_basic1_en'})
MATCH (e1:Ejercicio {idEjercicio: 'e101'})
MATCH (e2:Ejercicio {idEjercicio: 'e102'})
CREATE (e1)-[:PERTENECE_A {posicion: 1}]->(un);
CREATE (e2)-[:PERTENECE_A {posicion: 2}]->(un);
```

```
MATCH (e1:Ejercicio {idEjercicio: 'e101'}), (h:Habilidad {idHabilidad:
'h_present_simple'})
CREATE (e1)-[:REFUERZA]->(h);
```

```
MATCH (u:Usuario {idUsuario: 'u1'}), (e1:Ejercicio {idEjercicio: 'e101'}), (h:Habilidad
{idHabilidad: 'h_present_simple'})
CREATE (u)-[:REALIZA {
  fechaRealizado: datetime("2025-09-25T10:30:00"),
  resultado: 'incorrecto',
  tiempo: 40,
  intentos: 1
}]->(e1);
```

```
CREATE (u)-[:FALLA_EN {
  vecesFalladas: 5
}]->(h);
```

Una vez que se tienen los nodos creados y relaciones establecidas, si se desea consultar por los ejercicios para reforzar las habilidades donde un usuario tiene dificultades, se haría la consulta:

```
MATCH (u:Usuario {idUsuario: 'u1'})-[:ESTUDIA]->(idl:Idioma)
MATCH (u)-[:FALLA_EN]->(h:Habilidad)
MATCH (h)-[:DEL_IDIOMA]->(idl)
MATCH (e:Ejercicio)-[:REFUERZA]->(h)
MATCH (e)-[:PERTENECE_A]->(un:Unidad)-[:DEL_IDIOMA]->(idl)
RETURN DISTINCT e.idEjercicio AS ejercicio,
               e.nombre AS nombreEjercicio,
               h.nombre AS habilidad,
               f.vecasFalladas AS vecesFalladas,
               un.nombre AS unidad
ORDER BY f.vecasFalladas DESC;
```

Requerimiento 4

Para la implementación, se trabajará sobre los subsistemas de Motor de recomendaciones usando Neo4j y Gestión de usuarios y perfiles usando MongoDB.

Implementación de Gestión de usuarios y perfiles

Se crearon 4 colecciones en MongoDB:

- usuarios
- actividad_usuario:
- estadísticas_usuario
- logros_definicion

El modelo se encuentra en [Modelo de datos previsto](#)

La implementación se hizo con C# y Blazor.

El proyecto contiene las siguientes librerías y clases.

Domain (Capa de Dominio)

- Descripción de todas las clases de entidades
- Propiedades y propósitos de cada clase
- Subdocumentos y relaciones

DataAccess (Capa de Acceso a Datos)

- MongoContext y su funcionamiento
- Métodos de cada repositorio
- Operaciones bulk y características especiales

ObligatorioBDNR: aplicación de consola de .NET 8.0 con Blazor Server que implementa la Gestión de usuarios y perfiles.

Contiene:

- Aplicación de consola: ejecución en consola con salida de texto (mensajes de inicialización, seed data, validadores)
- Blazor Server: interfaz web interactiva accesible desde el navegador

Dentro de ObligatorioBDNR se encuentra una carpeta Services que contiene los siguientes servicios:

SeedService.cs: Pobla la base de datos con datos iniciales (seed data) para desarrollo y pruebas.

Funcionalidades principales

1. SeedAsync() — Datos básicos

- Crea 18 logros del sistema (ej: "Primera Lección", "Racha de 7 días", "100 XP").
- Crea 10 usuarios predefinidos con nombres reales:
- Juan Pérez, María García, Carlos Rodríguez, Ana Martínez, Luis Fernández, Sofía López, Pedro Sánchez, Laura Torres, Roberto Díaz, Carmen Ruiz
- Asigna a cada usuario:

- 1-5 cursos con unidades y niveles
- Logros obtenidos
- Actividades diarias
- Estadísticas (si es Plus)
- Relaciones de amistad bidireccionales
- Evita duplicados: verifica si los usuarios ya existen antes de crearlos.

2. Seed1000UsuariosAsync() — Datos masivos

- Genera hasta 100 usuarios adicionales (si hay menos de 100 en total).
- Para cada usuario:
 - 1-4 cursos con progreso variado
 - 5-8 actividades distribuidas en los últimos 30 días
 - 1-5 logros aleatorios
 - Estadísticas diarias (si es Plus, ~30% de usuarios)
 - 2-5 amigos con relaciones bidireccionales
- Usa inserción masiva (bulk insert) para mejor rendimiento.
- Verifica existencia antes de insertar para evitar duplicados.

3. Métodos auxiliares

- GenerarUnidadesCurso(): Crea la estructura de unidades y niveles de un curso.
- CalcularXpTotal(): Calcula el XP total sumando el XP de todos los cursos.

MongoValidatorService.cs: Configura validadores de esquema JSON Schema en MongoDB para garantizar la integridad de los datos a nivel de base de datos.

Funcionalidades principales

1. ConfigurarValidadoresAsync() — Configuración general

Configura validadores para 4 colecciones:

- usuarios
- actividad_usuario
- logros_definicion
- estadisticas_usuario

2. Validadores específicos

ConfigurarValidadorUsuariosAsync()

Valida la estructura de usuarios:

- Campos requeridos: _id, Email, Username, FechaCreacion, Autenticacion
- Email válido (regex)
- Username entre 3-50 caracteres
- Suscripcion.Tipo debe ser "Gratis", "Plus", "FREE", "PLUS" o null
- CompartirActividad debe ser "todos", "amigos", "nadie" o null
- Estructura de cursos, unidades y niveles
- Rangos válidos (ej: DiasActivosUltimos30 entre 0-30)

ConfigurarValidadorActividadUsuarioAsync()

Valida actividades:

- Campos requeridos: _id, IdUsuario, Fecha
- Estructura de Acciones con Tipo y Timestamp

- XpGanado ≥ 0

ConfigurarValidadorLogrosDefinicionAsync()

Valida logros:

- Campos requeridos: `_id`, `Nombre`, `Descripcion`, `XpRecompensa`
- Nombre entre 1-100 caracteres
- Descripcion entre 1-500 caracteres

ConfigurarValidadorEstadisticasUsuarioAsync()

Valida estadísticas:

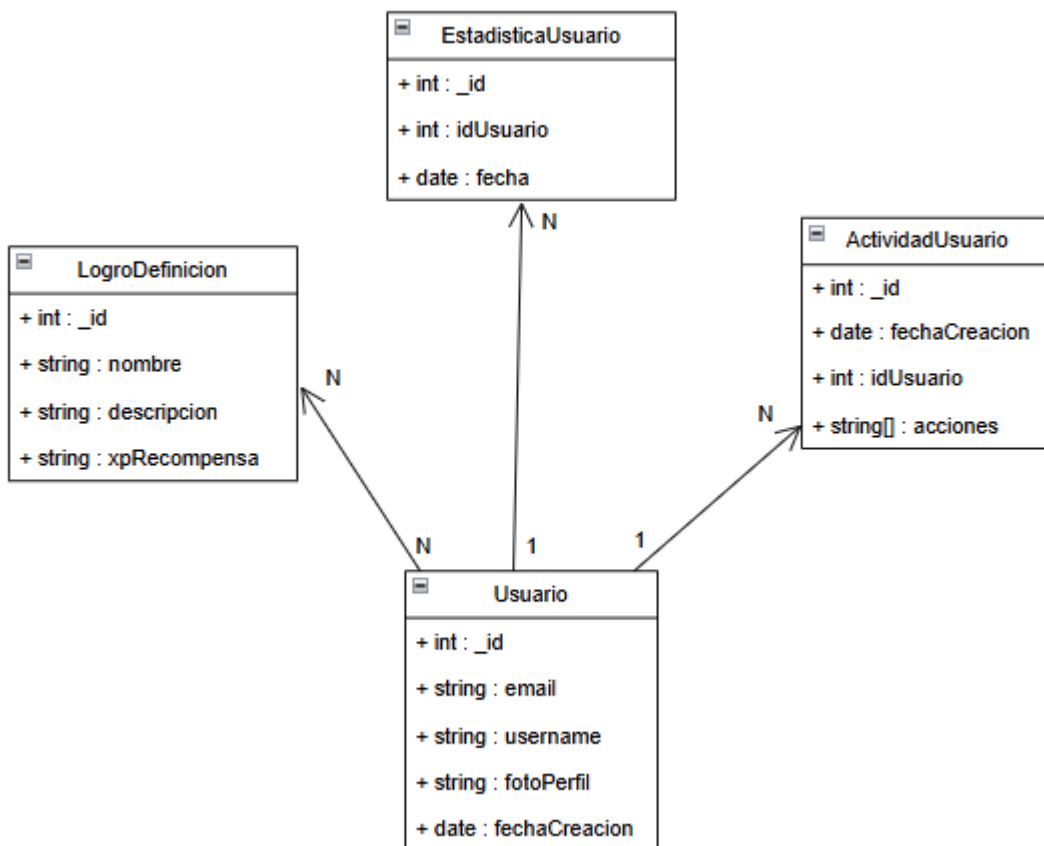
- Campos requeridos: `_id`, `IdUsuario`, `Fecha`
- Valores numéricos ≥ 0
- Estructura de `DetallePorCurso`

3. AplicarValidadorAsync() — Aplicación

- Si la colección existe: modifica el validador con `collMod`.
- Si no existe: crea la colección con el validador usando `create`.
- Nivel de validación: `moderate` (valida solo documentos nuevos/actualizados).
- Acción: `error` (rechaza operaciones que no cumplan el esquema).

Diagrama de mongo:

Cada colección tiene su sub documentación.

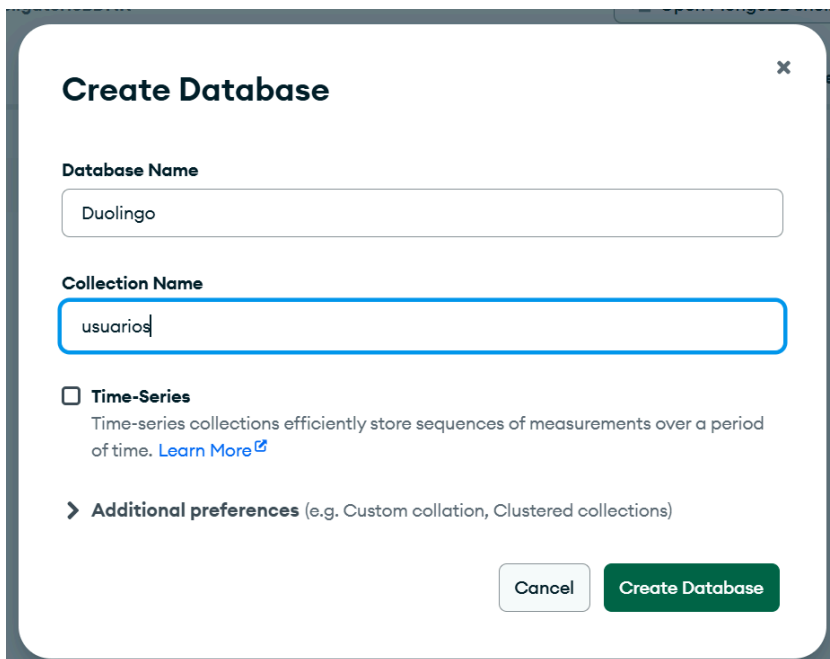


Flexibilidad del modelo

El modelo propuesto es flexible debido a su diseño orientado a documentos desacoplados, característico de MongoDB. La información del usuario se divide en colecciones independientes (usuario, actividad_usuario, logros_definicion, estadísticas_usuario), lo que permite que cada módulo evolucione sin afectar a los demás. El uso de arrays de subdocumentos (como la lista de cursos) permite representar estructuras dinámicas y expandibles sin necesidad de redefinir esquemas rígidos. Además, los datos de alto volumen, como actividades y estadísticas diarias, se almacenan fuera del documento principal, evitando su crecimiento excesivo y manteniéndolo liviano y eficiente. La arquitectura también facilita incorporar nuevos tipos de estadísticas, idiomas, logros o beneficios de suscripción sin alterar la estructura existente, garantizando escalabilidad y extensibilidad a largo plazo.

Pasos para ejecutar la implementación

Paso 1: crear una base de datos llamada Duolingo en una conexión que tenga como puerto localhost. Además, Mongo hace que crees una colección. Podemos crear la colección usuarios



Create Database

Database Name
Duolingo

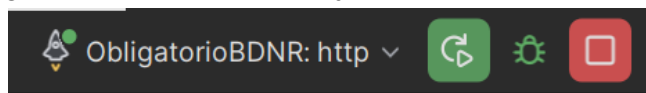
Collection Name
usuarios

☐ **Time-Series**
Time-series collections efficiently store sequences of measurements over a period of time. [Learn More](#)

> **Additional preferences** (e.g. Custom collation, Clustered collections)

Cancel Create Database

Paso 2: ejecutar la aplicación ObligatorioBDNR con http. Las colecciones y documentos se generan al momento de ejecutar la aplicación.



Run

ObligatorioBDNR: http

"C:\Program Files\JetBrains\JetBrains Rider 2024.1.1\plugins\dpa\DotFiles
ach-event-name=dpa.detach.19268 D:/BDNR/ObligatorioBDNR/ObligatorioBDNR/b
info: Microsoft.Hosting.Lifetime[14]
Now listening on: <http://localhost:5188>
info: Microsoft.Hosting.Lifetime[0]
Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
Content root path: D:\BDNR\ObligatorioBDNR\ObligatorioBDNR
warn: Microsoft.AspNetCore.HttpsPolicy.HttpsRedirectionMiddleware[3]

localhost:5188/login

Duolingo

Iniciar Sesión

Registrarse

Iniciar Sesión

Email

Contraseña

Iniciar Sesión

¿No tienes una cuenta? [Regístrate aquí](#)

Implementación de Motor de Recomendaciones

Para la implementación del Motor de Recomendaciones se creó una aplicación web con [ASP.NET](#) Core y Blazor en C# con conexión a la base de datos orientada a grafos Neo4j. La aplicación se conecta a Neo4j con el driver oficial Neo4j.Driver.

En esta aplicación, cuando se corre, se insertan datos de prueba ya que usa seeds para crearlos.

La idea de la aplicación es que se pueden crear usuarios, idiomas, unidades, habilidades, ejercicios y relaciones entre estos. Una vez que hayan datos, se puede consultar por recomendaciones en tiempo real según los 5 tipos detallados en el requerimiento 1:

- Recomendación basada en las dificultades del usuario
- Recomendación basada en la similitud de usuarios
- Recomendación basada en el contenido y la estructura del curso de cierto idioma
- Recomendación según el idioma estudiado por el usuario
- Consultas según diversos criterios

En la implementación se usan tecnologías como [ASP.NET](#) Core versión 8.0, Blazor, Bootstrap, para la base de datos Neo4j con el plugin Graph Data Science para utilizar algoritmos para las recomendaciones.

Así como se especificó en el requerimiento 3, se tienen los siguientes nodos:

- Usuario: representa a los estudiantes
- Idioma: idiomas disponibles
- Unidad: bloques de contenido con orden (posición)
- Habilidad: competencias específicas (gramática, vocabulario, etc.)
- Ejercicio: actividades que refuerzan habilidades

El plugin de Neo4j Graph Data Science (GDS), se utiliza para análisis de grafos y recomendaciones avanzadas. Se crean proyecciones de grafos y se aplican algoritmos específicos. Se usaron los siguientes algoritmos de GDS:

- Node Similarity para el segundo tipo de recomendaciones (basada en similitud de usuarios). Este algoritmo calcula la similitud entre usuarios según los ejercicios que realizaron.
- PageRank para la recomendación basada en el contenido del curso (el segundo tipo). Este algoritmo calcula e identifica ejercicios centrales en la estructura del curso para luego recomendarlo.
- Betweenness Centrality para el 4to tipo de recomendación (recomendación basada en el idioma que estudia el usuario). Este algoritmo identifica los ejercicios que conectan varias habilidades e idiomas.

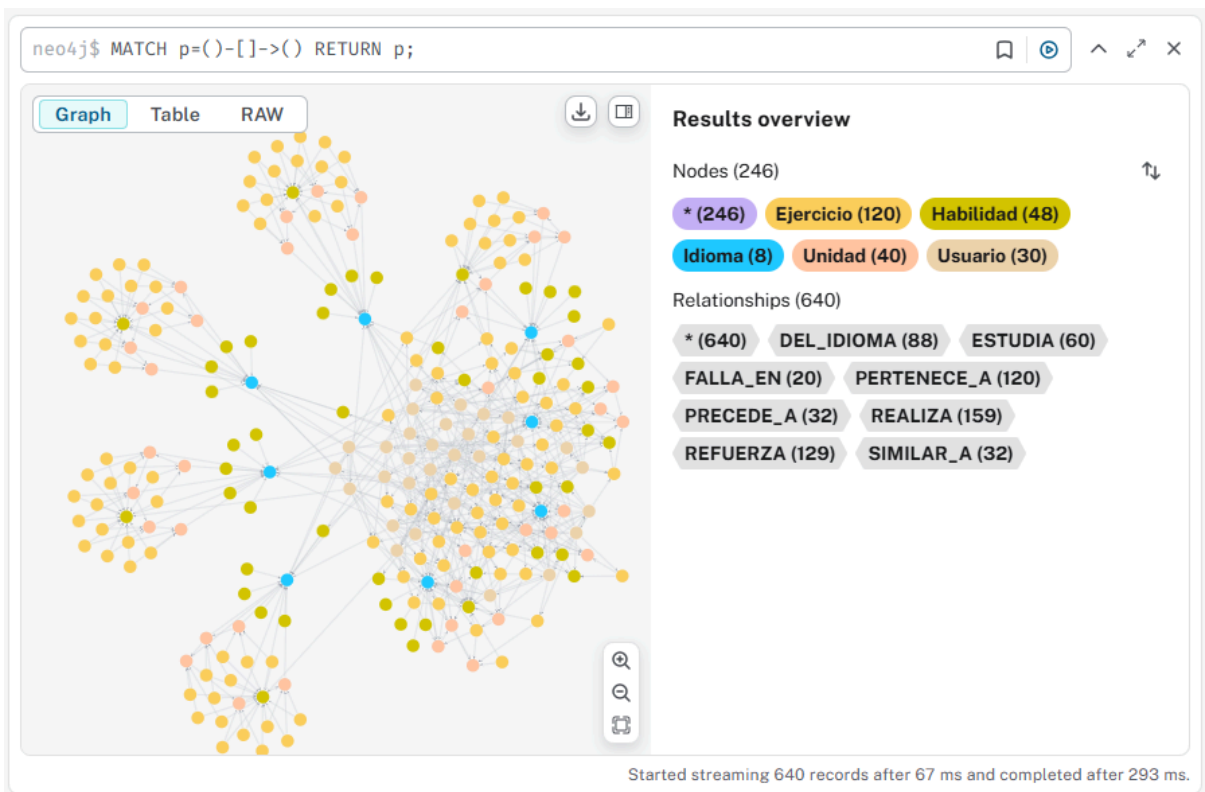
En caso de que por falta de grandes cantidades de datos, el algoritmo de GDS no funcione, se ejecuta un “plan b” realizado en Cypher para que de recomendaciones de todas formas.

Se logró implementar distintas funcionalidades que permiten cargar datos, que crean nodos o relaciones entre nodos y otra que permite consultar según usuarios y tipo de recomendación.

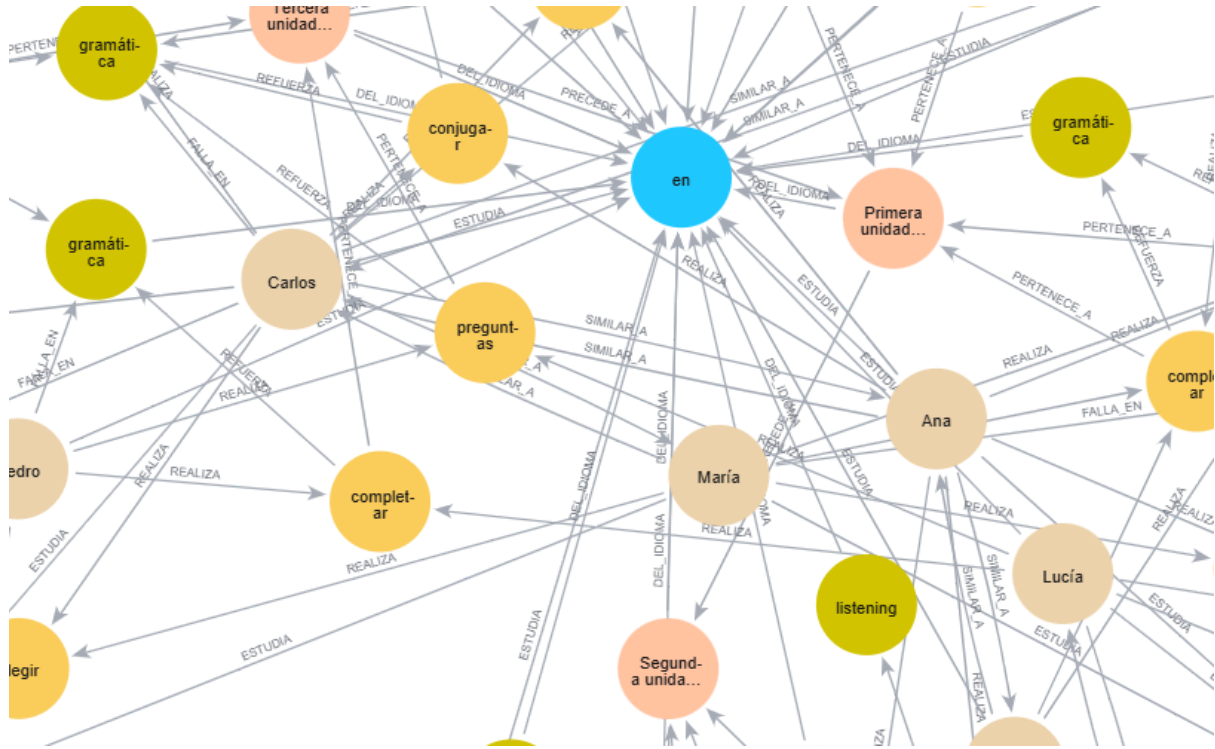
Cuando se dan recomendaciones de ejercicios, estos se muestran en una tabla de la siguiente manera:

Resultados (11 recomendaciones)							
ID Ejercicio	Nombre Ejercicio	Habilidad	Unidad	Idioma	Veces Falladas	Score	Tipo
e101	Crear oraciones present simple 1	Present Simple	Básico 1	Inglés	5	10,00	Recomendación combinada (GDS Multi-Algorithm)
e102	Completar present simple 1	Present Simple	Básico 1	Inglés	5	10,00	Recomendación combinada (GDS Multi-Algorithm)
e103	Traducir present simple	Present Simple	Básico 1	Inglés	5	10,00	Recomendación combinada (GDS Multi-Algorithm)
e301	Vocabulario comida - Matching	Vocabulario: Comida	Básico 1	Inglés	3	6,00	Recomendación combinada (GDS Multi-Algorithm)
e302	Vocabulario comida - Listening	Vocabulario: Comida	Básico 1	Inglés	3	6,00	Recomendación combinada (GDS Multi-Algorithm)
e501	Ser vs Estar - Ejercicio 1	Ser y Estar	Básico 1	Español	0	0,00	Recomendación combinada (GDS Multi-Algorithm)
e502	Ser vs Estar - Ejercicio 2	Ser y Estar	Básico 1	Español	0	0,00	Recomendación combinada (GDS Multi-Algorithm)
e201	Past simple - Verbos regulares	Past Simple	Básico 2	Inglés	0	0,00	Recomendación combinada (GDS Multi-Algorithm)
e202	Past simple - Verbos irregulares	Past Simple	Básico 2	Inglés	0	0,00	Recomendación combinada (GDS Multi-Algorithm)
e601	Pretérito - Conjugación	Pretérito	Básico 2	Español	0	0,00	Recomendación combinada (GDS Multi-Algorithm)
e401	Pronunciación - Vocales	Pronunciación	Intermedio 1	Inglés	0	0,00	Recomendación combinada (GDS Multi-Algorithm)

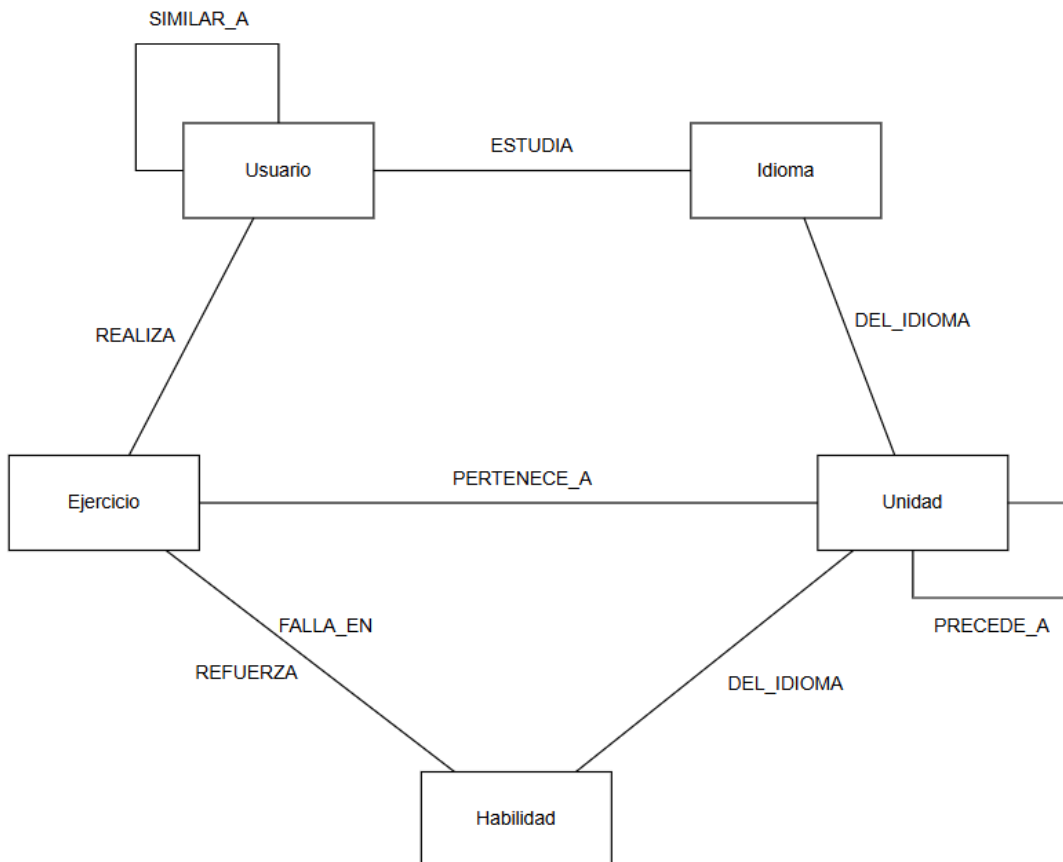
En Neo4j Desktop, los datos creados se ven de la siguiente forma:



Haciendo zoom se ve:



Un diagrama para ver las entidades (nodos) y sus relaciones es:



Cambios de la primera entrega

Dada la pregunta ¿Realizaría algún cambio de recomendación en alguno de los subsistemas que presentó en la primera entrega?,

No, porque cada subsistema que definimos en la primera entrega tiene un tipo de problema bastante claro y las tecnologías que elegimos siguen siendo las que mejor se ajusten a los casos de uso de los subsistemas. Las nuevas bases de datos que vimos podrían aparecer como complemento en ciertos escenarios puntuales pero no tienen una ventaja suficiente como para reemplazar las ya elegidas originalmente.

Bibliografía

- 1) [Apache Arrow | Apache Arrow](#)
- 2) [Format | Apache Arrow](#)
- 3) [Arrow Columnar Format — Apache Arrow v21.0.0](#)
- 4) [Reading and Writing Data — Apache Arrow Python Cookbook documentation](#)
- 5) [Architecture | Vertica 25.3.x](#)
- 6) [Partitioning | Vertica](#)
- 7) [Redis Copilot](#)
- 8) [What is Redis? - IBM](#)
- 9) [PostgreSQL 18.0 Documentation](#)
- 10) [Recommendation Engine & System Use Cases with Graph Databases](#)
- 11) [Neo4j-graph-recommendations-white-paper-EN-A4.pdf](#)
- 12) https://cassandra.apache.org/_/cassandra-basics.html
- 13) https://es.wikipedia.org/wiki/Apache_Cassandra
- 14) <https://docs.datastax.com/en/cassandra-oss/2.2/cassandra/cassandraAbout.html>
- 15) [Document Database - NoSQL | MongoDB](#)

Apartado de Inteligencia Artificial

En el desarrollo del presente proyecto se utilizaron herramientas de inteligencia artificial generativa como apoyo en distintas etapas del obligatorio. Su uso se orientó a complementar y facilitar el proceso de desarrollo.

Se utilizaron las inteligencias artificiales ChatGPT y Gemini buscando explicaciones en caso de no comprender la información técnica presentada en las páginas oficiales de motores de bases de datos como Arrow o Vertica.

También se utilizaron como una guía para comprender mejor lo pedido en la letra de la implementación de Apache Arrow, y el uso de ciertas librerías como os y pyarrow.

Se hizo uso de la IA generativa para comprender el funcionamiento de las nuevas tecnologías, como Neo4j, para resolver problemas complejos en la implementación y para crear los datos precargados.